PhyReAct: Agentic Physical Reasoning for Video Evaluation and Refinement

Wenzhuo Xu[§], **Yuchen Zhu**, **Chongjian Ge**, **Jing Shi**, **Jason Kuen**, **Tong Sun**, **Jiuxiang Gu**[§†]

Adobe Research
[§]Core Contribution, [†]Project Lead

Videos that look fluent are not necessarily physically reliable, yet most video evaluators collapse this judgment into a single scalar score that cannot be traced, audited, or corrected. We present PhyReAct, a physically grounded instance of the mainstream ReAct agentic paradigm: a strong reasoning model runs an explicit thought–action–observation loop in which reasoning compiles the prompt into typed physical obligations, and observations gate and scope which operators fire next. The actions invoke a suite of frozen low-level physical experts—detection, counting, tracking, segmentation, depth, OCR, and audio—that return only measurements, never verdicts. A deterministic layer composes these measurements into a three-valued state (**supported**, **contradicted**, or **unknown**, surfaced as *plausible*, *implausible*, or **abstain**) together with full provenance, yielding a human-auditable trace rather than an opaque number. We anchor evaluation in a 1,500-clip corpus of human-annotated flaw records that localize real generation failures in prompt reference, space, and time. On the current development core—used during system development and not held out—plan-grounded ReAct execution recovers more localized flaws than one-at-a-time per-claim verification, at a cost of roughly one additional operator call per clip; precise figures and their uncertainty are reported in the body. The resulting evidence traces are designed as an interface for context-training self-evolution, operator-library growth, external-knowledge retrieval, and critic-to-generator refinement through a control channel. Closing this optimization loop remains future work.

Date: July 2026

1 Introduction

Text-to-video systems now produce clips with striking appearance and smooth local motion, yet visual fluency is not physical reliability. A generated video can preserve style and identity while omitting a requested object, reversing the order of two events, violating a contact outcome, or following a trajectory inconsistent with the described scene. Recent benchmarks expose this gap between perceptual quality and physical faithfulness [5, 6, 41, 45]. For evaluation that could eventually drive correction, however, a scalar quality score is not enough: it cannot say which prompt-conditioned obligation failed—over entities, events, motion, contact, space, quantity, and time—where in the clip it failed, or on what evidence the judgment rests. What is missing is not a better number but a decision that names the violated obligation and exposes the evidence behind it.

We approach physical evaluation as physically grounded reasoning in the reason–act–observe style. ReAct established the general pattern of interleaving a model's reasoning with tool actions and observations of their results [71]. PhyReAct is a grounded instance of that pattern specialized to physical verification: a reasoner compiles the prompt into *typed physical obligations* and an executable measurement plan, and observations returned by physical operators then gate and scope which later operator calls are valid and where in the video they run. Concretely, the current system operates as a plan-then-execute member of this family—reasoning declares the obligations and their measurement plan up front, and observations activate, skip, or localize the already-declared measurements rather than inventing new tool sequences at run time. What is special about PhyReAct is not the control flow, which is inherited from mainstream reason–act–observe agents, but the *physical grounding* of that flow: frozen low-level physical operators (detection with LLMDet, counting with CountGD, point tracking with TAPNext, segmentation with SAM 3, monocular depth, optical character

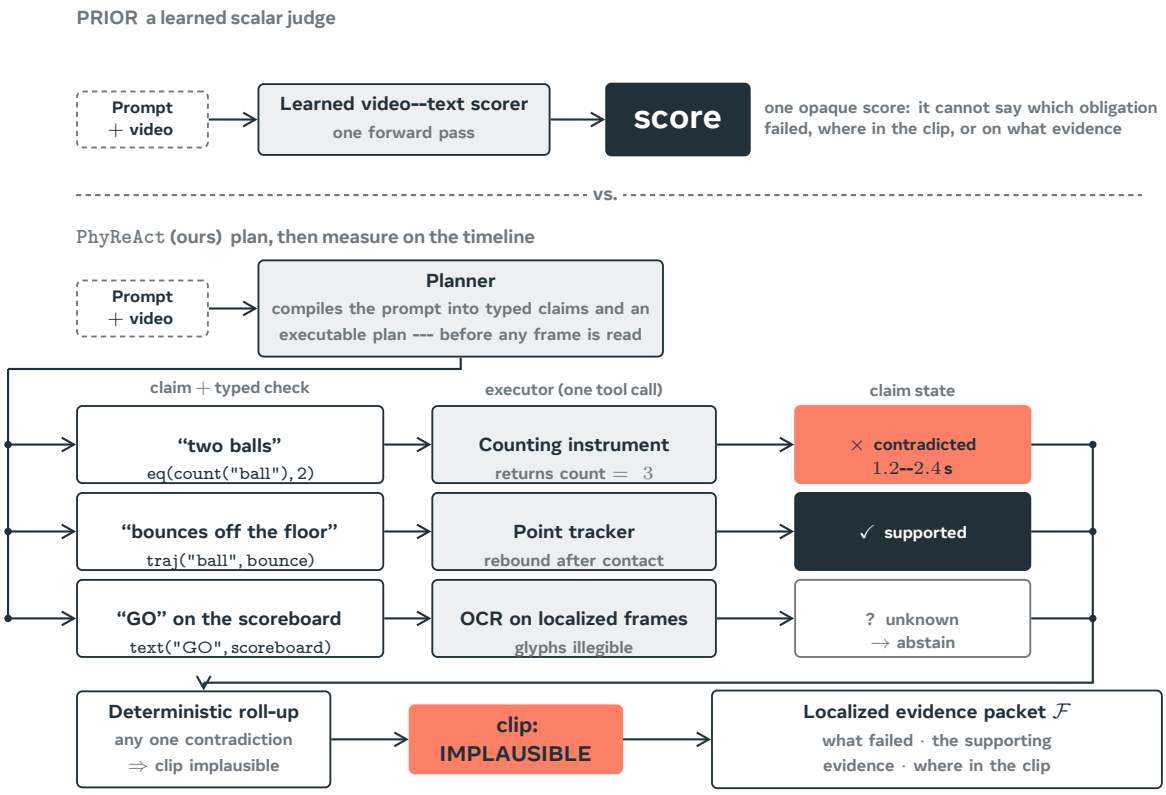


Figure 1 `PhyReAct` turns an opaque quality score into an auditable verdict. A learned video–text scorer maps a prompt and clip to a single number that cannot say which prompt obligation failed, where in the clip, or on what evidence. `PhyReAct` instead runs a reason–act–observe loop: reasoning compiles the prompt into typed physical obligations and an executable measurement plan, frozen physical operators return localized measurements, and fixed rules combine them into **supported**, **contradicted**, or **unknown** claim states, a *plausible*, *implausible*, or **abstain** clip decision, and a localized evidence packet recording what failed, when, and on what basis. The claims and windows shown are an illustrative schematic of the output contract, not a measured run.

recognition, and audio event detection) return measurements rather than verdicts, a strong reasoning model supplies the semantics, and deterministic rules compose the measurements into a three-valued decision with full provenance. This lineage is a design choice rather than a claim of autonomous replanning: the run-time freedom is confined to the gate-and-scope semantics just described.

Four roles divide the labor. A *text-only planner* reads the whole prompt and writes the typed obligations and their measurement plan before any frame is seen. A *video-aware semantic verifier* reads timestamped frames densely for predicates such as existence and event occurrence. *Physical operators* supply targeted measurements—counts, tracks, masks, depths, rendered text, and sound intervals. A *deterministic aggregator* then applies typed predicates and three-valued composition, so that no learned component emits the final label. The video's timeline structures execution as a conditioned evidence chain: an event must be observed before its time interval can support anything else (occurrence gates timing), that confirmed interval restricts the frames a more expensive operator examines (localized scope), and temporal and spatial relations are then computed from the resulting intervals by fixed rules. Earlier observations therefore determine which later measurements are valid and where they are taken, rather than leaving evidence as an unordered bag of model answers. The output is a claim-level trace that a person can read, audit, and hand off to human-in-the-loop review (figure 1).

Evaluating such a critic requires ground truth at the same level of detail. We therefore assemble a corpus of 1,500 generated clips with 2,582 human-written flaw records, each of which quotes the exact prompt fragment that was violated and adds a rationale, a severity, a confidence, and, where the annotator could supply them, a temporal span and an object track. These human labels anchor evaluation in real failures and are the only

ground truth in this work; the critic's own verdicts are never treated as such. Unlike a single clip-level score, the annotations let each allegation the critic makes be matched to the particular failure a person marked.

The same structured trace also defines an interface to refinement: a contradicted claim names what should change, and its localized evidence indicates when and where an intervention is needed. As a controllable testbed for this direction, we render MuJoCo simulations as depth or silhouette control videos for a Wan 2.2-VACE generator, so that geometry and motion come from the simulator while appearance comes from the text prompt [27, 58, 59]. This gives a control channel through which critic feedback could later update *how a clip is generated*. In this report the critic and the generator are studied separately; closing the critic-to-generator loop remains future work.

Reading physical evaluation as a reason–act–observe framework also makes several extensions natural, which we state as design directions rather than completed results. Because the plan is written as a program over operators, the operator library can grow with experience into a reusable, cost-annotated toolset [47, 67]. Because every trace is inspectable and version-controllable, traces and their feedback could accumulate as human-readable prior context and let the system evolve without updating weights [25, 77]. And because a loop that feeds only its own traces back into its own context is internally closed, admitting external or social knowledge would break that closure—but only under a candidate-and-pruning discipline that guards against context pollution [25]. None of these extensions is implemented or measured here; they describe where the framework points.

Our contributions are threefold. First, we present `PhyReAct`, a physically grounded reason–act–observe framework for auditable video evaluation that compiles a prompt into typed physical obligations, gates and scopes a set of frozen physical operators over the video timeline, and composes their measurements into a three-valued, fully attributable verdict. Second, we introduce a 1,500-clip corpus of human-annotated, prompt-grounded flaw records together with a paired flaw-level evaluation that isolates the benefit and cost of plan-based execution against one-at-a-time per-claim verification. Third, we connect this critic to a simulation-conditioned generation testbed, establishing a concrete control-channel interface for future critic-guided refinement. As a fourth, forward-looking element, the framework's inspectable traces are designed to support operator-library growth and weight-free context-training self-evolution, which we leave to future work.

On the direction of the measured comparison, plan-based, timeline-aware evidence collection recovers more human-marked flaws than isolated per-claim verification, at the price of about one extra model or tool call per clip. This comparison is made on a 149-clip development core that is recall-only, single-annotator, and *not* held out—the system was developed against it—so the result characterizes the current system rather than held-out generalization; Section 5 reports the precise figures and their uncertainty.

2 Physics-Guided Video Generation

2.1 Backbone and control mechanism

The backbone is Wan 2.2-VACE-Fun-14B, a two-expert mixture-of-experts video-diffusion model with a native control branch, run through the `WanVACEPipeline` interface; because the two expert stacks exceed the memory of a single 80 GB accelerator, the experts are CPU-offloaded during generation. Generation takes four inputs: the pre-computed control video, a mask, the text prompt, and one scalar conditioning strength. Nothing about appearance is taken from the simulation: the control video decides where surfaces are and how they move, while the prompt alone decides material, colour, lighting and setting (figure 2).

The mask decides which parts of the frame are held fixed and which are synthesised. Dark mask regions mark content to be preserved as given; bright regions mark content to be generated under the guidance of the control. Inside the backbone, the mask splits the control video into a preserved part and a generated part, which are encoded separately and concatenated into the control latent stream. We pass an *all-white* mask, so the preserved part is empty and every pixel of every frame is synthesised: the depth field is used purely as guidance and no region of it is copied through into the result. This is the mechanism that lets a grey depth render drive a photorealistic clip without tinting it grey.

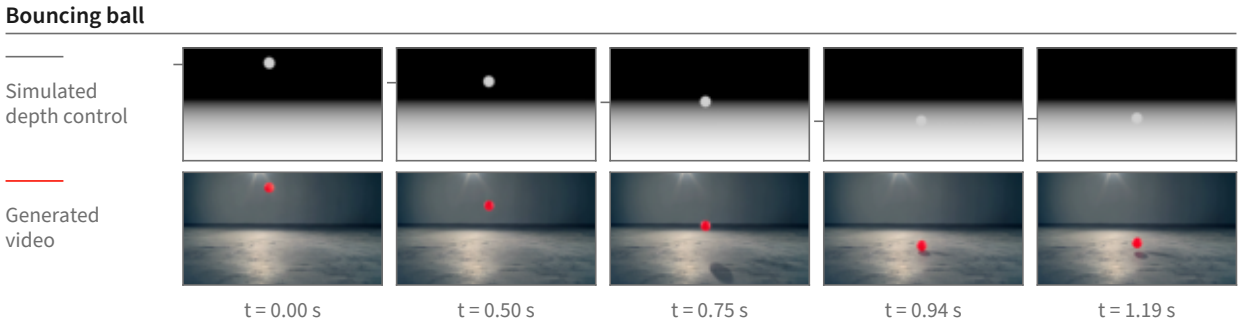


Figure 2 Physics-guided generation, shown on real frames of one bouncing-ball clip. **Top:** the depth control video rendered from the rigid-body simulation, the only geometric input the backbone receives. **Bottom:** the clip generated from it, where the text prompt supplies appearance alone. Both rows are sampled at the same five timestamps of the same 5.06 s, 16 fps, 848×480 clip; the timestamps are chosen by tracking the ball and taking the turning points of its vertical path, so the columns show the drop, the floor contact and the rebound rather than five evenly spaced instants. Tracking the ball independently in each row, the generated path follows the control's with a correlation of 1.0000 horizontally and 0.9997 vertically over all 81 frames, and a median positional disagreement of 0.7 pixels across the frame and 4.7 pixels down. The tick beside each control frame marks the ball's measured centre height; the frames themselves are unaltered.

The conditioning strength is the scalar by which the control latent stream is added into the denoising stream at each of the backbone's control layers. One value applies uniformly to every control layer; a per-layer schedule is also accepted by the interface but is not used here. Turning it down weakens how tightly the generated motion is bound to the simulated motion without changing anything else about the request.

Appearance comes only from text. The prompt names the target scene in the ordinary way ("a bright red rubber ball bouncing on a polished concrete studio floor"), and a single fixed negative prompt—shared by every clip—pushes away from the rendered look the control video literally has, listing cartoon, CGI, render, grayscale and flat shading among the things to avoid. No appearance reference image is supplied. Sampling is deterministic given the seed, and the seed is recorded per clip.

Two properties of this arrangement are worth stating plainly. First, the backbone consumes a control video that was *computed*, not estimated: no monocular depth predictor sits in the loop, so the geometry the generator follows is exactly the simulator's geometry, with no estimation error of its own. Second, this is a control-branch mechanism rather than image-to-image editing: it is neither an SDEdit-style noised re-synthesis [40] of a rendered source nor a copy of source pixels, which is what separates it from conditioning schemes that carry a synthetic source's look into the output.

2.2 Rendering the control signal

One simulation, several registered views. A scene is described as data—a list of moving bodies with initial positions and velocities, a list of static surfaces, and a camera placed by a look-at rule—and the same engine turns any such description into a simulation. Dynamics are integrated in MuJoCo [58] under gravity at $9.81\,\mathrm{m/s^2}$ with a one-millisecond solver step, a fourth-order integrator, and an elliptic friction cone. Output frames are produced on a coarser clock: the requested scene duration is divided into the frame budget, and the solver is advanced by that many sub-steps between frames. At every output frame, and from the *same* solver state, three renderers are queried in turn—a shaded colour view, the depth buffer, and the object-segmentation buffer. Because they read one state, the resulting videos are registered pixel for pixel, so a depth control and a silhouette control for the same scene describe the same geometry and can be substituted for one another without re-simulating.

From depth buffer to control video. The depth buffer holds distance from the camera. It is turned into a control video by an inverted, clipped affine map: distances are mapped so that near surfaces are bright and far surfaces dark, which is the polarity monocular depth estimators produce and therefore the polarity

the control branch was trained to read. The mapping's endpoints are robust percentiles—the first and the ninety-seventh—of the depth values in the scene, computed after discarding the far-clip constant that the background sky writes into the buffer, so that a single distant constant cannot compress the whole range into a few grey levels. Formally, for depth frame D_t over pixel domain Ω and $t \in \{0, \dots, T-1\}$, let

$$\begin{aligned} \mathcal{D} &= \{D_t(u) : t \in \{0, \dots, T-1\},\ u \in \Omega,\ D_t(u) \neq d_{\text{far}}\}, \\ d_- &= Q_{0.01}(\mathcal{D}), \qquad d_+ = Q_{0.97}(\mathcal{D}), \\ C_t(u) &= 1 - \text{clip}\left(\frac{D_t(u) - d_-}{d_+ - d_-},\, 0,\, 1\right). \end{aligned} \tag{1}$$

Here d_{far} is the renderer's background constant and C_t is the grayscale control frame before boundary smoothing. If $d_+ = d_-$, the depth map is degenerate and the silhouette control described below is used instead.

Why the mapping is fixed for the whole clip. The endpoints are computed once over all frames of the clip, not per frame. This is a substantive choice, not a detail: it makes the assigned grey level a function of distance alone, so the same distance reads as the same grey in every frame, and a change in brightness therefore means a change in distance. If the endpoints were recomputed per frame—which is the convention monocular depth estimators follow, since they emit one relative map per image—the grey assigned to a distance would depend on the depth distribution of that particular frame. A body holding its distance while other content enters or leaves the view would then change brightness, and the control branch has no way to distinguish that from the body actually moving toward or away from the camera. Fixing the mapping across the clip removes that ambiguity by construction. A per-frame variant is still rendered alongside the fixed-mapping one so the two can be compared directly on the same simulation.

Boundary treatment. A simulator's depth buffer has ideal, hard edges: a silhouette boundary is a one-pixel step. Estimated depth maps, which is what the control branch has seen, do not look like that—their boundaries are soft. Each grayscale frame therefore receives a small symmetric Gaussian blur before encoding. Because the kernel is symmetric, the half-intensity contour of an edge stays where it was, so the smoothing changes how the boundary is presented without displacing the geometry it represents.

Silhouettes, and when they replace depth. The segmentation buffer labels each pixel with the geometry that produced it. Collecting the labels that belong to the simulation's moving bodies and painting them white on black yields a silhouette control: the moving bodies' outlines over time, with no distance information at all. Depth is the default because it carries both outline and distance ordering. It stops being informative when a scene is flat—when every object rests on one near-horizontal surface at nearly the same distance from the camera, the depth field is almost constant across the objects and their surroundings, and the control has little geometric contrast to offer. Measured on the rendered controls, the depth separation between a moving body and the surface immediately around it is about twenty grey levels for a ball bouncing on a floor and about a hundred for a ball arcing through open air, but only about fourteen for a rack of balls on a tabletop. For those flat layouts we substitute the silhouette, which restores a maximum-contrast figure-and-ground signal; the price is that distance ordering between the objects is discarded, and the generator has to recover it from the prompt and from ordinary scene priors.

A timing property of the renders. The frame budget and playback rate are fixed—eighty-one frames written at sixteen frames per second—while the simulated interval differs per scene. The clip therefore plays the simulated motion back stretched in time, by factors between roughly 1.8× and 2.3× for the scenes used here. The trajectory shapes and the contact events are the simulation's; their absolute rate in the finished clip is not, and a downstream measurement of speed or duration must be read against the render clock rather than the simulator's.

Relation to structure conditioning generally. Driving synthesis from an explicit geometric control rather than from text alone follows the lineage of ControlNet for images [73] and motion and structure control for video [27, 61]. What differs here is the provenance of the control: it is not estimated from a photograph

Setting	Value
Backbone	Wan 2.2-VACE-Fun-14B (two-expert MoE), diffusers `WanVACEPipeline`, bf16
Memory	experts CPU-offloaded (two-expert weights exceed an 80 GB accelerator)
Control signal	MuJoCo z-buffer depth (globally normalized); object silhouette for flat scenes
Conditioning	control video + all-white mask + text prompt; no reference image
Resolution	848×480 (flow-shift 3.0)
Frames / rate	81 frames at 16 fps
Sampler / steps	UniPC, 50 steps
Guidance scale	5.0
Conditioning scale	1.0 (swept over $\{0.5, 0.8, 1.0\}$)

Table 1 Deployed generation configuration, from the generation scripts. A small checkpoint provides a pre-flight sanity gate and is not used for final clips.

or drawn by hand but computed from a physics solver, so the structure the generator is asked to follow is physically consistent by construction.

2.3 Pipeline and configuration

Table 1 lists the operating configuration. Generation runs as a staged GPU job: control videos are rendered, a small checkpoint provides a fast sanity gate, the 14B checkpoint generates the clips (data-parallel when more than one GPU is available), and the clips are streamed to storage.

2.4 Design choices

Two choices define the deployed recipe. **Native control rather than source re-synthesis:** a control branch consuming a pre-computed depth video keeps the generator from importing the rendered source's synthetic look, which re-synthesizing the simulation frames directly does not avoid. **No reference image:** the pipeline accepts an optional appearance reference, but the deployed recipe omits it, because including it degraded single-object trajectories under inspection.

2.5 Scene library and coverage

The two cases above are single rigid bodies because those are the scenes whose trajectory can be checked against a closed-form path. The scene set itself is much wider: 1,314 scenes across 66 kinds. Before any of them was rendered photorealistically, 652 of the underlying simulations were watched and 591 judged to show the intended physical behaviour clearly enough to be worth generating from; that review is of the simulations, not of the generated video. figure 4 shows twelve of those kinds as real generated frames—cloth draping and crumpling, a flag in wind, a swinging rope and a hanging chain, grains pouring into a heap and draining through a funnel, a ball dropped into a pit of balls, a loaded boat, a cascade down a slope, toppling dominoes and a chaotic two-arm linkage. figure 5 shows three of them the same way figure 3 shows a ball: the simulated depth control above, the generated clip below, at matched timestamps.

One limit should be read off these figures explicitly. Everything here comes out of the same rigid-body engine, so the deformables are that engine's particle-and-link primitives: cloth is a sheet of particles joined by springs, a rope or chain is a short row of linked segments, sand and marbles are many small solid bodies in contact, and floating is a buoyancy force on a solid body in a still fluid volume. None of it is a fluid solve, none of it is smoke, and none of it is a finite-element soft body. The scene captions name the primitive rather than the phenomenon for exactly this reason.

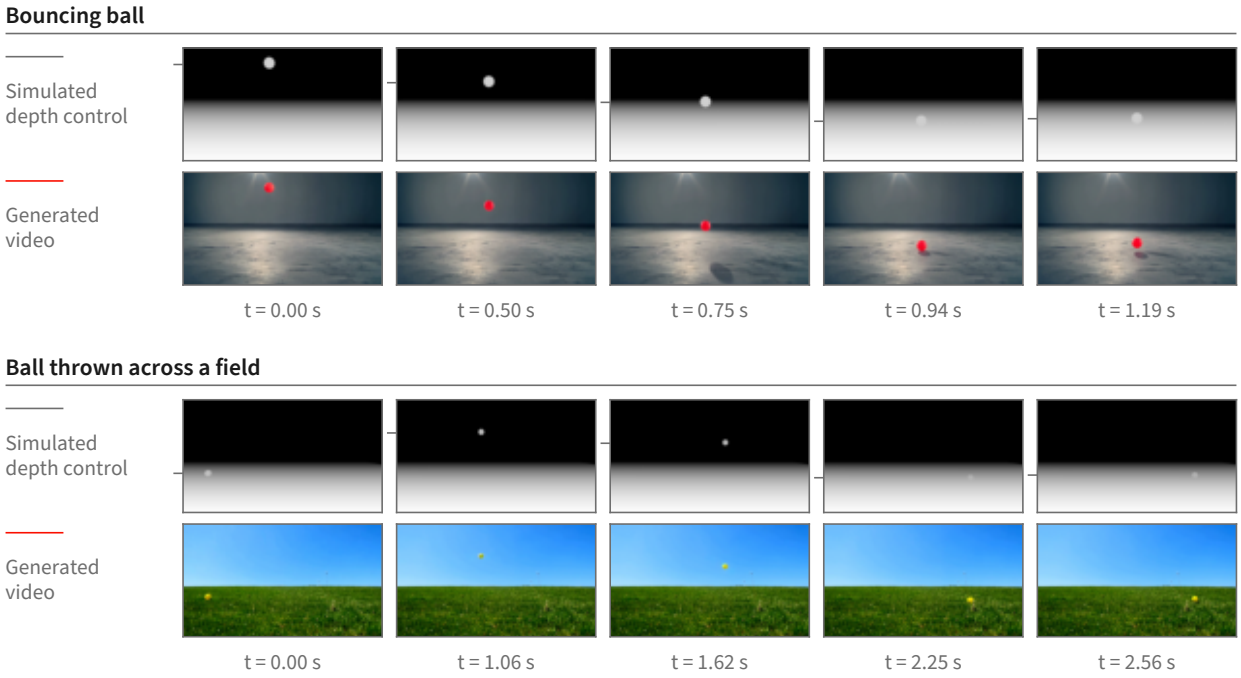


Figure 3 The same mechanism across two scenes with very different requested appearance—an indoor studio and an outdoor field—each shown as its depth control above and the resulting generated clip below. Each scene has its own timestamps, chosen from the turning points of that scene's tracked vertical path so every column carries motion. The appearance changes completely with the prompt while the motion stays the simulation's: for the thrown ball the generated and control paths agree to a correlation of 1.0000 horizontally and 0.9998 vertically, with a median disagreement of 0.7 pixels across and 1.1 pixels down, over the 63 frames in which the ball is measurable in both before it leaves the frame.

2.6 What the generator achieves

Single-object scenes—a bouncing ball, a projectile arc, a swinging pendulum—render cleanly, with the generated object following the simulated trajectory (figure 3). Multi-object interaction, such as a billiards break, remains open: neither the deployed backbone nor the alternates tried against it produce a clean multi-object collision.

These judgments are qualitative. They rest on visual inspection of the output clips and on a color-blob trajectory tracker that checks whether the generated object follows the control's path. No learned or quantitative video-quality score—FVD, a human-preference model, or a critic score—is applied to this line.

2.7 When the control can be released

The control does not have to be applied for the whole of denoising. A diffusion model builds a video over a sequence of denoising steps that starts from almost pure noise and ends at a clean frame, and the control signal is added in at every one of those steps. That raises a question with both a practical and a scientific side: if the control is switched off partway through, does the motion it has already induced survive to the finished clip? Practically, the steps after the release are free of the control, so the model is free to invent appearance there. Scientifically, the earliest release point that still preserves the motion is the point at which the model has *committed* to that motion.

Method. We pick a step, keep the control at full strength for every step before it, and set the control's contribution to exactly zero from that step onward. Nothing else changes: same simulation, same control video, same prompt, same sampler, same seed. The comparison is always against the same clip generated with the control never released, so the release point is the only difference between the two.

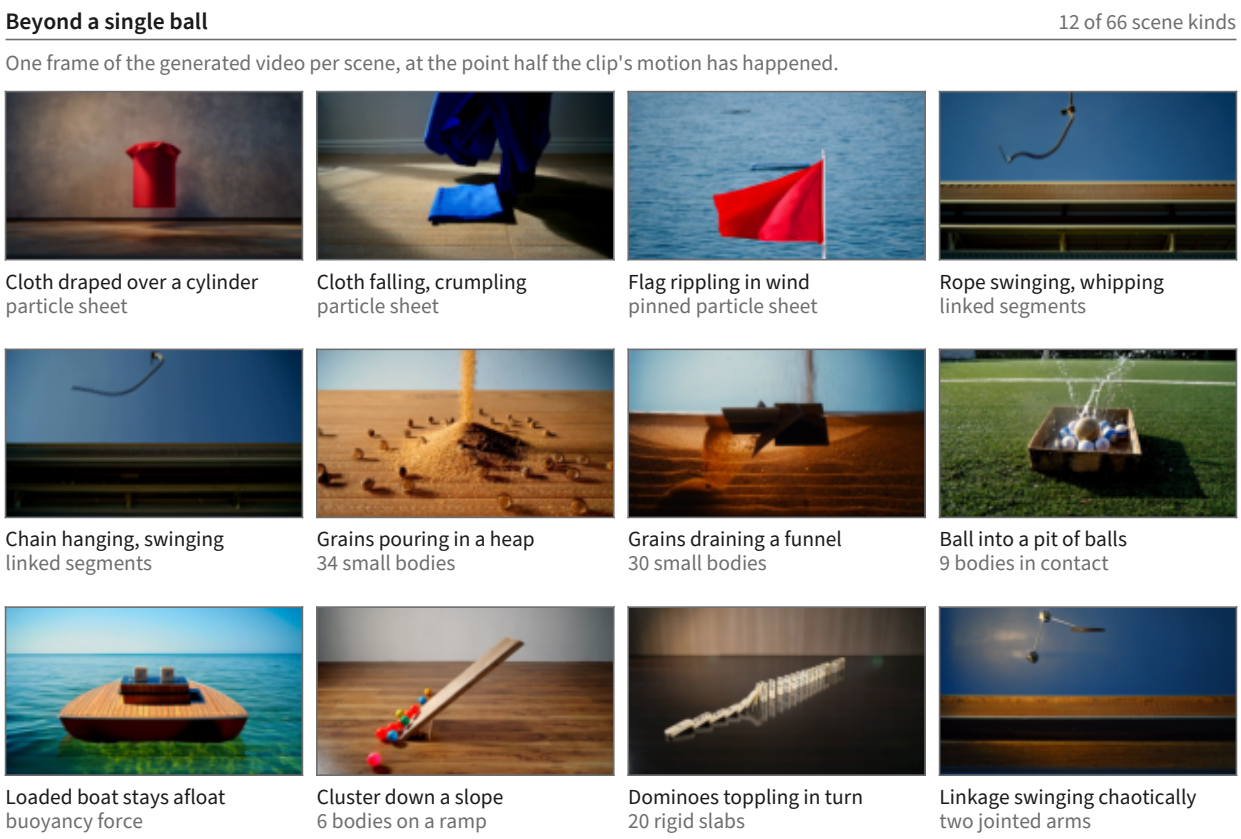


Figure 4 Twelve of the 66 scene kinds, one real generated frame each. The frame shown is computed, not picked: for every clip the frame-to-frame change is accumulated over the whole clip and the tile is the frame by which half of that total change has happened, which lands mid-drape, mid-swing or mid-pour rather than on the first frame. The second caption line names the physics primitive, and every body count in it is the scene's own parameter. The simulation each clip was generated from passed the pre-generation review described in the text; the generated clips themselves were checked by eye for this figure, which is a weaker guarantee than a systematic review. Two candidate tiles were cut after tracking their control renders showed the phenomenon their label would have claimed does not occur in the clip.

The release point is reported as a noise level, not a step number. A step index is not a property of the model; it is a property of the sampling schedule we happened to choose. The schedule maps each step to a noise level, and that mapping changes when the number of steps or the schedule's shape changes. Concretely, on the schedule used here, step 8 of 40 sits at a noise level of $\sigma = 0.952$; on a differently-shaped 40-step schedule the same step 8 sits at $\sigma = 0.923$, and the noise level 0.952 has moved to step 5. Reporting "released at step 8" would therefore describe our schedule rather than the model, so every release point below is given as the noise level σ at which the control was switched off, with the step index alongside it only as the label of the run.

What is measured. Both the control video and the generated clip are tracked frame by frame, giving the object's path in each. *Retention* is how much of the simulated motion the generated clip keeps: the agreement between the generated path and the control's path, measured separately across the frame (horizontal) and up and down (vertical), because those two directions do not behave the same way. Agreement is a correlation over the frames in which the object is located in both videos, so a value near 1 means the generated object moved the way the simulation did. Two windows are reported, because a single whole-clip number hides where a failure lives: the *airborne* window, up to the frame at which the simulated object first reaches the ground, and the *after-contact* window from that frame to the end. The contact frame is not hand-set per scene; it is read off the control's own vertical path as the first return to the floor after the object's highest point. Alongside retention we count two failure modes. A *collapse* is a clip whose horizontal agreement falls below 0.7, meaning the sideways motion no longer resembles the simulation's. A *phantom* is the specific

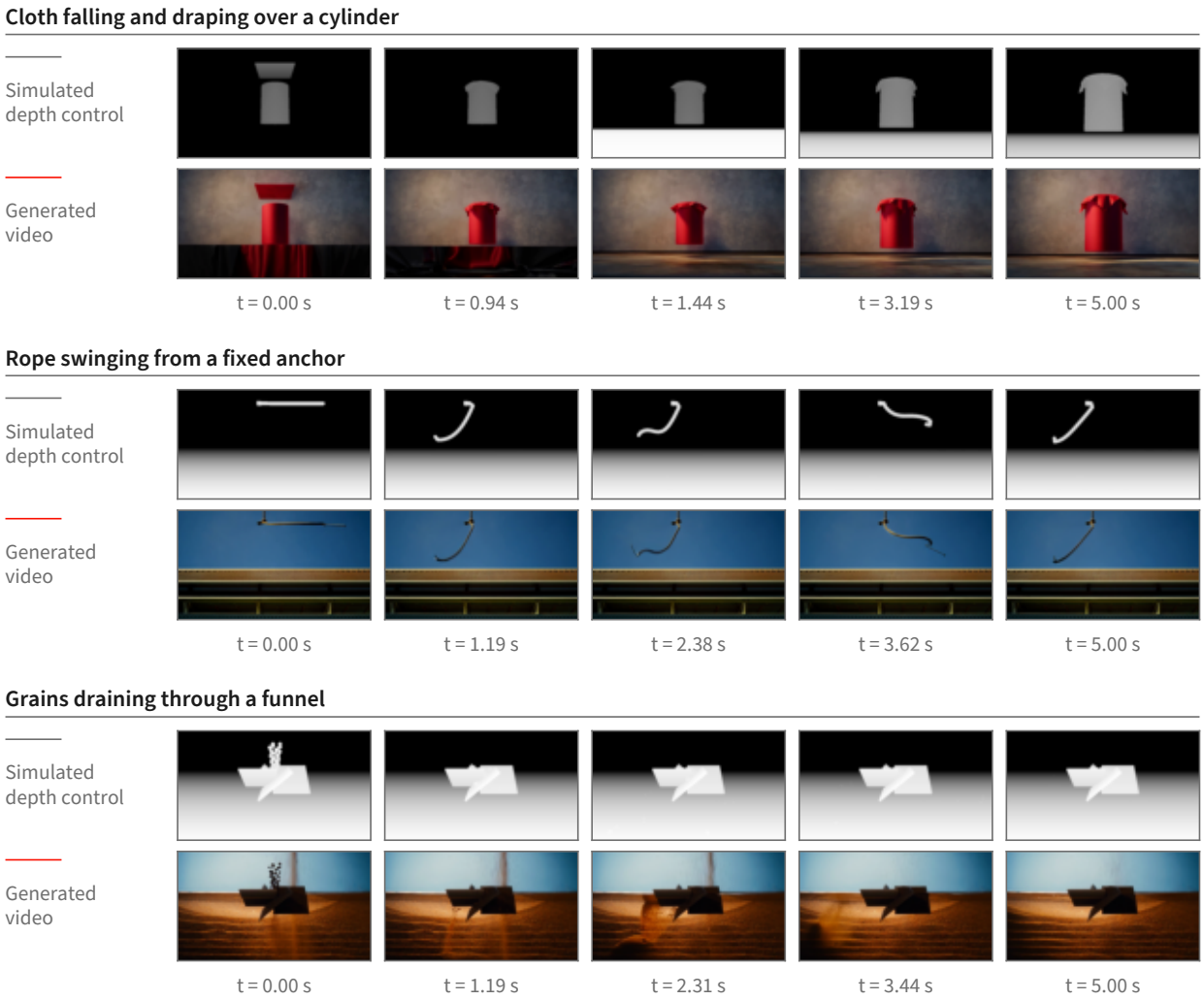


Figure 5 Three deformable and granular scenes shown as figure 3 shows a ball: the simulated depth control above, the generated clip below, at matched timestamps. Columns are placed at equal steps of the clip's accumulated change so no column falls in a static stretch. These three cases are the ones whose depth control still shows its object in every column—in a depth picture an object that settles onto the floor takes on the same grey as the floor behind it and disappears from the control, so how much of each control frame is distinguishable from its background is measured, and cases whose control goes blank are not shown as pairs.

disagreement in which the whole-clip number reads as broken (below 0.7) while the airborne window alone reads as clean (above 0.9) — the signature of a clip whose flight is right and whose ending is wrong. Both thresholds were fixed before scoring.

Result: the flight survives an early release; the ground contact does not. Figure 6 shows the measurement. Two scenes were repeated over 16 seeds each at one release point, $\sigma = 0.952$, in two variants (with and without a reset of the sampler's internal history at the release), against the never-released baseline: 96 clips in total, 16 per cell. Figure 6 shows that comparison. Releasing the control at $\sigma = 0.952$ leaves the airborne flight of the thrown ball essentially untouched — per-seed horizontal agreement drops by 0.003 relative to never releasing (95% confidence interval 0.001 to 0.004, $n = 16$ paired seeds), which is a real but negligible difference. Over the whole clip the same release costs 0.13 horizontally (95% CI −0.02 to 0.28, $n = 16$) and 0.18 vertically (95% CI 0.07 to 0.29, $n = 16$). The gap between those two windows is the finding: what an early release breaks is not the arc but what happens after the ball reaches the ground. Scoring only the frames after the simulated contact, the generated ball's horizontal motion runs *opposite* to the simulation's on 9 of 16 seeds

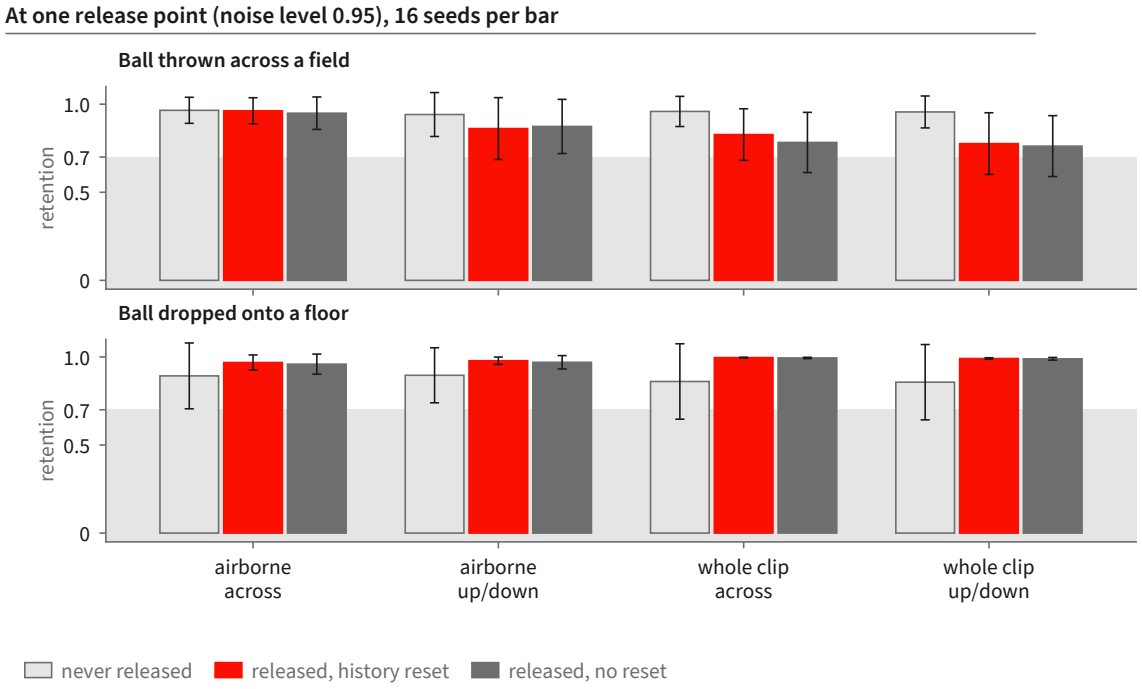


Figure 6 The seeded measurement at the single release point $\sigma = 0.952$, 16 seeds per bar, error bars showing 95% confidence intervals of the mean. Retention is the agreement between the generated object's tracked path and the simulation's, so 1.0 is "moved the way the simulation did". The three bars in each group are the never-released baseline and the two release variants, which differ only in whether the sampler's internal history is also reset at the release. The airborne window is indistinguishable from never releasing; the whole-clip number is not, and the direction that suffers is the horizontal one. Grey band marks the region below the pre-registered 0.7 break line.

when the control is released at $\sigma = 0.952$, and on 0 of 16 when it is never released — the simulation rolls the ball forward and the generator, cut loose, rolls it back. The whole-clip collapse count follows: 3 of 16 seeds at $\sigma = 0.952$ versus 1 of 16 never released, and the phantom count is 2 of 16 versus 0 of 16, so the collapses that do appear are predominantly ending failures on clips whose flight was clean.

Result: a later release is clean, and the vertical direction is the robust one. Sweeping the release point across six settings on a single seed for three scenes gives the shape of the curve (figure 7). For the thrown ball, releasing at $\sigma = 0.952$ gives an after-contact horizontal agreement of -0.63 (the ball travels the wrong way) with the airborne window still at 1.00; releasing at $\sigma = 0.903$ or later restores the after-contact window to 0.98 or above, and it stays there through the latest release point tested, $\sigma = 0.555$. The same sweep on a ball dropped onto a floor never breaks at all — after-contact horizontal agreement is 1.00 at every release point including the earliest, and the vertical agreement, which is the informative direction for a straight-down drop, is lowest at the earliest release (0.78 at $\sigma = 0.952$) and rises to 0.99 or above once the release is at $\sigma = 0.903$ or later. A ball rolling off a ramp holds 0.99 or better at every release point in both directions, but only a third of its frames are measurable at all, so it constrains little. Across all three scenes, vertical motion — the direction gravity supplies — is markedly more robust to an early release than horizontal motion, which the simulation alone specifies.

A denser sweep, and what it does not settle. A third run sweeps three release points on the thrown ball at three seeds and two guidance strengths, 18 clips, and scores them against the control with a segmentation-based tracker rather than a colour tracker. It reproduces the direction of the effect — airborne horizontal agreement is 1.00 (mean over the 6 clips at $\sigma = 0.952$, 95% CI 0.998 to 1.000), while after-contact horizontal agreement at that same release point averages 0.69 with a confidence interval spanning zero (-0.04 to 1.43, $n = 6$) and is negative on one clip; at $\sigma = 0.903$ and $\sigma = 0.833$ the after-contact window recovers to 0.99 and 1.00. But it

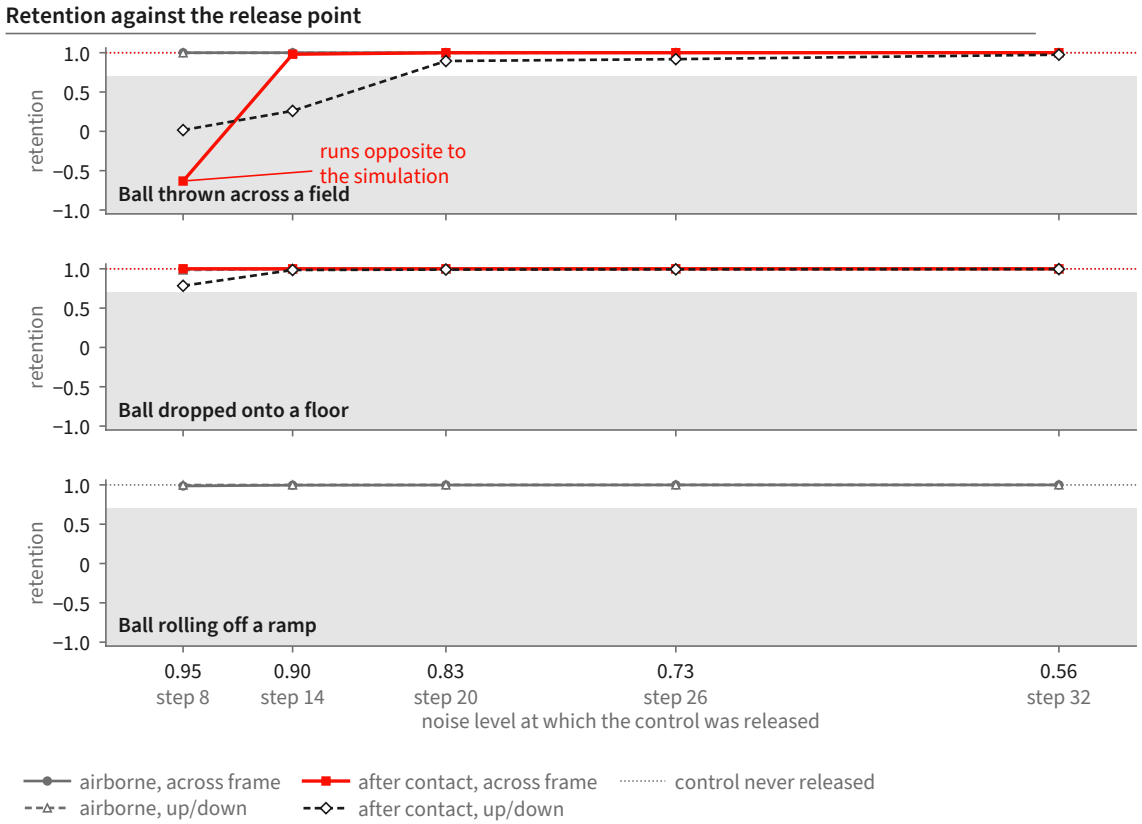


Figure 7 The release-point sweep: one clip per point, three scenes, scored separately over the airborne window and the window after the simulated object first reaches the ground. The horizontal axis is the noise level at which the control was switched off, high noise on the left; the step index of each run is printed underneath as a label only, because the same step index sits at a different noise level under a different schedule. The thrown ball's after-contact horizontal motion is the one curve that breaks, and it breaks only at the earliest release: it runs *opposite* to the simulation at $\sigma = 0.952$ and is restored from $\sigma = 0.903$ onward. Its airborne flight is unaffected at every release point. Across all three scenes, vertical motion — the direction gravity supplies — is markedly more robust to an early release than horizontal motion, which the simulation alone specifies. Grey band marks the region below the pre-registered 0.7 break line.

also shows the ceiling on this measurement: at the latest release point the tracker loses the object entirely on 3 of 6 clips, so those cells rest on 3 clips, not 6. On inspection those are clips in which the generated ball is genuinely tiny and low-contrast against the grass, not clips in which the tracker mis-fired, but either way the confidence intervals at the late release points are too wide to place the transition sharply.

Scope. These numbers characterize one conditioning route. The runs condition the backbone by carrying a rendered source video through the denoising process and releasing it partway, rather than through the dedicated control branch of section 2. The question and the noise schedule are the same in both cases, but the release point measured here is a property of the source-carrying route, and its transfer to the control branch is untested.

The scenes are single moving rigid bodies—a ball thrown across a field, a ball dropped onto a floor, a ball rolling off a ramp—at one resolution and frame count. Deformable and granular scenes (section 2.5) are outside the study, because the tracker cannot follow their objects as a single path. Statistical depth is uneven: one release point carries 16 seeds, while the shape of the curve across release points rests on one seed per point for three scenes and three seeds per point for a fourth.

One factor is not isolated. In one of the two variants the release is accompanied by a sampler-history reset,

and the study does not separate the two. The variants are statistically indistinguishable on every window and direction measured—the largest per-seed difference is 0.04 horizontally over the whole clip, 95% CI −0.04 to 0.13, $n = 16$—which is what one expects if the reset does nothing once the motion has committed, but it does not establish that it never matters.

3 A Human-Annotated Video-Flaw Benchmark

The benchmark is a corpus of 1,500 AI-generated clips in which a person watched each clip against its prompt and wrote down every place the video failed to deliver what the text asked for. Every clip carries at least one such record. The generating model is not recorded in the data, and no clip is attributed to a specific system. The human-written fields are the ground truth throughout this report.

3.1 What a flaw record contains

A flaw is a localized record with six fields. Four are always present: the **violated fragment**, the words of the prompt the video fails, quoted verbatim; a free-text **rationale**; a **severity**; and the annotator's **confidence**. Two are present when the annotator could supply them: a **time span**, the first and last frame in which the failure is visible, and a **box track**, boxes at the start, middle and end of that span, one per object involved, in normalized coordinates. Table 2 gives each field's measured coverage across the evaluation core, and figure 8 shows one record in full. Each clip additionally carries an overall severity and a short list of the objects, actions and relations its prompt calls for.

Two coarse tags support grouping and reporting: a *modality* tag saying whether the complaint concerns what is seen, what is heard, or both, and a *type* tag naming the dominant kind of complaint from a fixed ten-item list. Neither is written by the annotator. Both are assigned by three language models classifying each flaw from its quoted fragment and rationale at temperature zero under a fixed seed, resolved by majority; a three-way disagreement resolves to "both" for modality and "other" for type, and a flaw receiving no usable vote falls back to a frozen keyword rule. They are labelled machine-assigned wherever they appear and are never treated as ground truth. Annotation is single-annotator: the corpus carries one annotation per flaw, so no inter-annotator agreement figure is available.

3.2 How the evaluation set is derived

Selection runs in three recorded stages, and the denominator of every later measurement depends on them. Table 3 gives the counts and figure 9 the same derivation as a sequence of steps.

Stage one, clip selection. A clip enters the pool if its video is retrievable and it shows signs of dynamics. The dynamics test is a fixed vocabulary of motion words—falling, bouncing, rolling, colliding, pouring, toppling—applied to the prompt and, separately, to each flaw's rationale and quoted fragment; a clip is kept when its prompt contains one, or one of its flaws does, or it has a time-windowed flaw against an action-typed object. The test is a keyword rule and selects for prompts and complaints that *talk about* motion, which is a proxy for, not a guarantee of, a flaw whose adjudication needs physical reasoning. This stage keeps 606 of the 1,500 clips, carrying 1,171 flaws.

Stage two, flaw selection. Within those clips, a flaw is a usable detection target only if a critic seeing only pixels could in principle find it. Flaws tagged "seen" or "both" are kept and purely audible complaints dropped, which removes 64 flaws and leaves 1,107 as the full evaluation set.

Stage three, the frozen core. A core of 150 clips with 306 flaws is drawn for routine measurement. The draw is deterministic and its contract recorded: a fixed seed; strata crossing overall clip severity, flaw count per clip and clip duration; largest-remainder proportional allocation across strata; within each stratum, candidates ordered lexicographically then shuffled once by a generator whose call sequence is fixed; and a replacement rule drawing from the same stratum for any clip that cannot be downloaded or decoded. A continuity-focused stratum of 29 clips and 85 flaws sits inside the core and is exempt from replacement. Every clip is downloaded and decoded before the core is written, and the selection, strata and replacements are stored with the set.

Field	What it holds	Coverage
violated fragment	the words of the prompt the video fails, quoted verbatim; median 8 words	306/306
rationale	free text saying what went wrong instead; median 25 words	306/306
severity	1–5; the distribution is centred at 3 and 4	306/306
confidence	1–5; 239 of 306 are marked 4 or 5	306/306
time span	first and last frame in which the failure is visible	237/306
box track	boxes at the start, middle and end of that span, one per object involved, in normalized coordinates	208/306

Table 2 The six fields of a flaw record and how often each is filled across the 150-clip core. The first four are always present; timing and boxes are not, and a flaw carrying neither is one whose complaint is about the clip as a whole or about something that never appears.

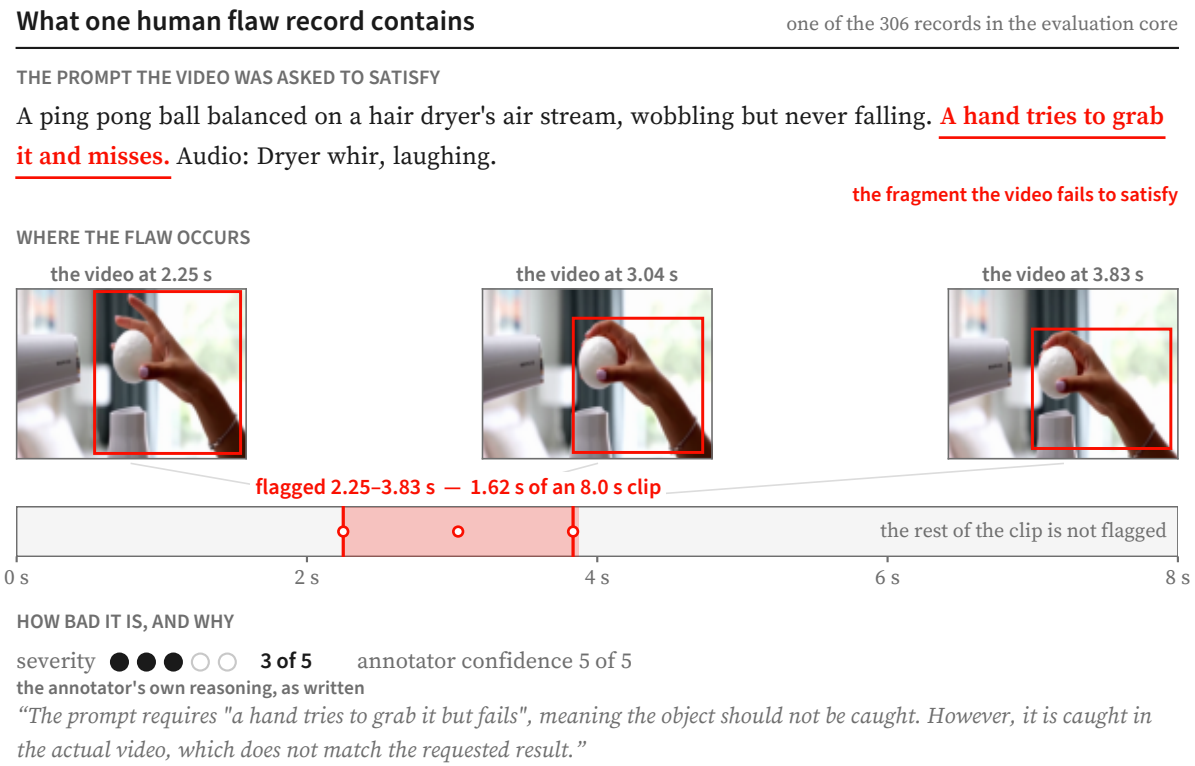


Figure 8 One human flaw record, shown in full. The annotator quotes the prompt fragment the video fails to satisfy, bounds the interval in which the failure is visible, boxes the objects involved at the start, middle and end of that interval, and writes a severity, a confidence and a rationale in their own words. Frames and boxes are the annotator's; the flagged window is 1.62 s of an 8 s clip. Nothing about the critic's output appears here.

The core is an *optimization set*: system and prompt decisions are made against it, so results on it measure fit to that core, and a generalization claim requires a set that has not influenced system design. A separate pool of 711 annotated clips is held disjoint from both frozen sets for any learning that consumes this data.

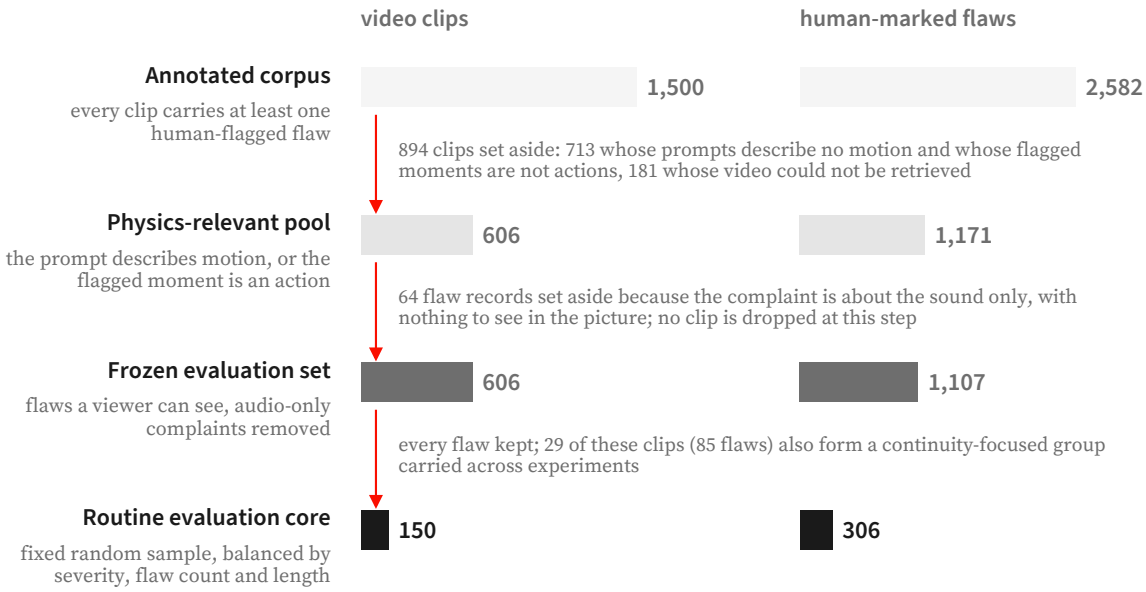


Figure 9 How the evaluation set is derived from the annotated corpus. Clips are removed only at the first step; the second step removes flaw *records* whose complaint is audible but not visible, leaving the clip count unchanged. The routine core is a fixed random sample balanced by severity, flaw count and clip length. A further 711 annotated clips are kept aside for training and share no clip with either frozen set.

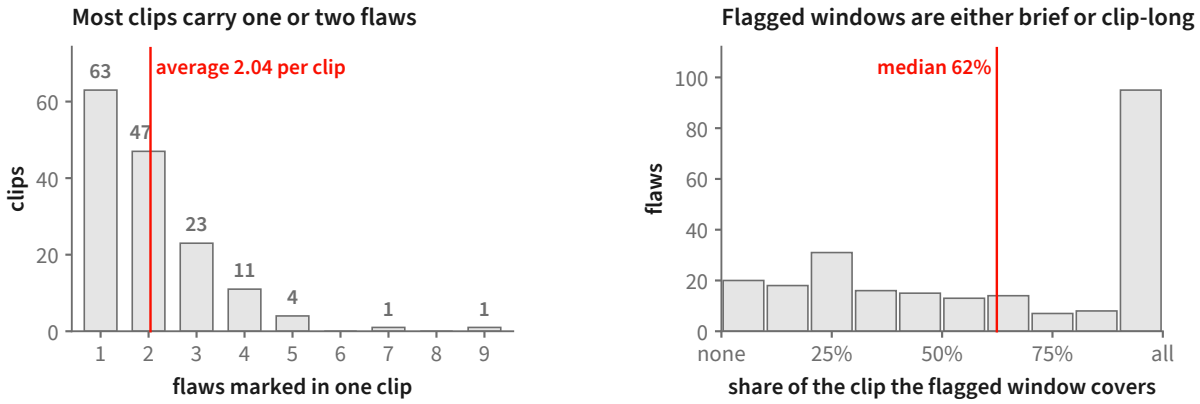
Set	Clips	Human flaws
Full corpus (all AI-generated, all annotated)	1,500	2,582
Physics-relevant pool	606	1,171
Full evaluation set (frozen)	606	1,107
Routine evaluation core (frozen)	150	306
continuity stratum within the core	29	85
Disjoint training pool (excludes the frozen sets)	711	—

Table 3 Benchmark statistics, from the dataset manifests. The full evaluation set drops a small number of audio-scoped flaws relative to the physics-relevant pool. The training pool excludes every clip in the frozen evaluation sets.

3.3 What is in the evaluation core

The core is 150 clips, median 8.0 s long (range 5.0–10.3 s), mostly at 24 fps (118 of 150; the rest at 16, 25, 30 or 32). The prompts are not one-line captions: a median prompt is 42 words and the longest 188, and many specify a shot list, a count, an ordering, an on-screen string and a sound in one paragraph. Sixty-six of the 150 clips carry flaws of more than one kind, so a single number per clip cannot say which of them a detector found.

Its 306 flaws, a median of two per clip, group by machine-assigned type as: actions that do not happen or happen wrongly, 139; wrong counts, 36; wrong or missing objects, 30; garbled on-screen text, 25; wrong spatial arrangement, 19; wrong ordering in time, 17; wrong attributes such as colour or material, 16; camera and style, 10; physical motion, 7; and a residual 7. The largest category is whether the requested action occurred at all, so this is not a physics benchmark in the narrow sense. Counting, text and spatial arrangement together account for 80 flaws, each a question a general-purpose model answers poorly and a dedicated measurement



150 clips of the routine evaluation core, 306 human-marked flaws. The right-hand panel uses the 237 flaws that carry a start and an end frame; the remaining 69 were marked without one. Of those 237, 57 fit inside a quarter of their clip and 95 run almost its whole length; the median flagged window lasts 5.00 s.

Figure 10 Two properties of the routine evaluation core that shape how a detector must behave. **Left:** most clips are flagged in only one or two places, so a detector that raises many complaints per clip will mostly be raising ones no annotator marked. **Right:** flagged windows are bimodal — many failures are confined to a small part of the clip while a comparable group runs almost its whole length — so both a brief localized check and a whole-clip check are needed.

answers well.

Of the 237 flaws carrying a time span, the flagged window covers a median of 62% of the clip, and the distribution is bimodal: 57 are confined to a quarter of the clip or less, and 95 run for at least nine-tenths of it (figure 10). A detector needs both a way to look closely at a moment and a way to judge a whole clip.

3.4 Scoring conventions

Every clip carries at least one flaw. With no clean clips, recall is directly measurable and precision is not: a detector that flags everything scores perfectly on recall alone, so results on this set are reported alongside the rate at which accusations land on a real flaw.

Scoring is over *distinct* flaw spans. Twelve clips record the same failure two or three times under different identifiers—eighteen duplicate entries across the core—and an accusation can be credited to at most one flaw, so identical spans on a clip count as one flaw. The core's 306 entries are therefore 288 distinct flaws. A run that covers a subset of clips carries its own share of both counts; section 5 states them.

3.5 Relation to existing video benchmarks

The closest annotated resource is Physion-Eval [74], which pairs generated clips with real-world reference processes and provides expert reasoning traces containing temporally localized glitches, a physical-failure category, and an explanation. VideoPhy-2 [6] instead has three annotators rate semantic adherence and physical commonsense, and mark predefined physical rules as violated, followed, or undetermined. Per-dimension suites such as VBench [26] and T2V-CompBench [56] primarily support comparison among generators on fixed prompt collections.

Our corpus targets a complementary, prompt-grounded detector protocol. Each record attaches a failure to the exact words of the generation prompt and permits a time span and box track, so an allegation can be matched to a particular annotated failure rather than only correlated with a clip-level score. Unlike Physion-Eval, this protocol does not require a matched real-world reference video; unlike VideoPhy-2, it uses open-vocabulary flaw statements rather than a fixed candidate-rule list. These choices make claim-level miss attribution possible in section 5, but they do not improve annotation reliability: the present corpus

is single-annotator and contains only flawed clips, whereas VideoPhy-2 provides redundant judgments and majority resolution.

4 PhyReAct: Auditable Physical Verification

4.1 PhyReAct as a physically grounded ReAct instance

PhyReAct inherits the control structure of the ReAct paradigm [71]: a text-only reasoner interleaves *reason*, *act*, and *observe* steps, and the result of each action conditions what the agent does next. The reasoning step compiles the evaluation prompt into a set of *typed physical claims* together with an executable measurement plan—the physical obligations that the video must satisfy for the prompt to hold. Each *act* step dispatches a frozen low-level physical operator (detection with LLMDet [15], counting with CountGD [3], point tracking with TAPNext [78], segmentation with SAM 3 [8], depth, OCR, or audio event grounding with FlexSED [20]) or a dense-frame semantic verifier over the sampled frames. The *observe* step consumes the returned measurement and uses it to gate whether a claimed event occurred, to scope which frames and entities subsequent operators attend to, and—via fixed, deterministic rules—to compute the physical relations that the typed plan declared. As in classic ReAct, tool outputs steer the trajectory rather than a fixed pipeline; unlike a free-form ReAct rollout, the trajectory is anchored to obligations that were declared before any pixels were read.

The one deliberate departure from unconstrained ReAct is a *grounding* choice made for auditability and physical correctness, not a rejection of the paradigm. Because a free-running agent that invents new tool sequences mid-rollout is difficult to audit and easy to lead into unfalsifiable narratives, PhyReAct pushes the open-ended reasoning to the front: the planner compiles and *statically validates* the typed measurement plan before it observes the video, in the spirit of plan-before-act agents [28, 64] and code-as-plan systems that emit an executable program of visual primitives [19, 57]. At execution time, observations activate, skip, or localize nodes that were already declared, rather than authorizing the model to synthesize arbitrary new tool chains. Crucially, the operators only return *measurements*; the verdict is left to a deterministic composition over those measurements. This separation—operators measure, fixed rules adjudicate—is what distinguishes PhyReAct from prompting a model to read the frames and pronounce a conclusion directly, and it is what lets us surface a human-traceable chain of evidence behind each **supported**/**contradicted**/**unknown** decision. PhyReAct thus sits in the plan–execute branch of the ReAct family: a physically grounded instance in which reasoning compiles the prompt into typed physical obligations and observations gate and scope the operators that discharge them.

CoT-as-program and a reusable operator library. Viewed this way, the compiled closure of each typed check—its input scoping, the operator invocations it triggers, and the deterministic rule that turns their measurements into a relation—behaves like a self-contained, reusable *operator*. Nothing about a given closure is specific to a single video: it can in principle be named, encapsulated, and versioned, and it already carries the cost accounting we record in the provenance trace (operator calls and accelerator time per node). This is the natural substrate for a training-free procedural operator library that grows and is retrieved across tasks, as envisioned by procedural-memory and self-evolving-reasoning proposals [47, 67]. We stress, however, that this is a framing of the design, not a claim about the current system: as implemented, PhyReAct treats every clip independently and performs *no* cross-clip reuse, and no operator ever persists or is retrieved from a prior evaluation. Curating the compiled closures into a versioned, cost-annotated operator library—and demonstrating that reuse improves recall or reduces spend—is future work (section 6.6); we neither run nor report any such experiment here.

4.2 Problem formulation and system overview

A generated clip is paired with the prompt p that requested it. We represent the decoded video by its own container clock,

$$\mathcal{V} = \{(I_t, \tau_t)\}_{t \in \mathcal{T}_\mathcal{V}}, \qquad \mathcal{T}_\mathcal{V} = \{0, \dots, T_\mathcal{V} - 1\}, \qquad 0 \le \tau_0 < \cdots < \tau_{T_\mathcal{V}-1} \le D_\mathcal{V}, \tag{2}$$

where I_t is frame t, τ_t is its timestamp, and $D_\mathcal{V}$ is the clip duration. The explicit range on t separates a frame index from time in seconds and also permits nonuniform decode timestamps. An audio track, when present, is

addressed on the same clock.

`PhyReAct` has four roles. A *text-only planner* decomposes the full prompt and writes a verification program before seeing any frame. A *video-aware semantic verifier* reads timestamped frames densely for predicates such as existence and event occurrence. *Specialist instruments* return targeted measurements such as counts, tracks, masks, depths, text, and sound intervals. Finally, a *deterministic aggregator* applies typed predicates and three-valued composition. The planner and semantic verifier may use the same served checkpoint, but they are separate calls with different inputs and responsibilities.

The planner produces exact prompt-grounded claims and a surface plan,

$$\mathcal{C}(p) = \{c_i = (s_i, \phi_i)\}_{i=1}^{N_p}, \qquad s_i \sqsubseteq p, \qquad (\mathcal{C}, \widetilde{\Pi}) = \Pi_\theta(p), \tag{3}$$

where s_i is a verbatim span of p and ϕ_i is the checkable assertion attached to it. The relation $s_i \sqsubseteq p$ is enforced, not inferred after execution. Types belong to the plan's operations and checks rather than to an unverifiable free-form claim label.

Figure 11 summarizes the resulting dataflow. Planning is *global with respect to the prompt*: the naming pass sees the whole prompt and all later claim groups share that vocabulary. Execution can be *local in time*: a declared interval may gate or scope a later declared action. This is the plan–execute branch of the reason–act–observe family (section 4.1): observations activate, skip, or localize nodes already in the program, but they do not search over alternative plans or invent a new tool recipe during execution.

4.3 Prompt claims and text-only plan synthesis

Claim extraction preserves both the exact source span s_i and its normalized assertion ϕ_i. A first pass over the whole prompt assigns stable identifiers to every distinct entity and to every separately asserted event occurrence. The claims are then planned in groups of at most four. Each group sees the full prompt, the shared names, and only its assigned claims; its local identifiers are renamed mechanically before all groups are assembled. This construction prevents two groups from silently referring to the same entity by different names or to different occurrences by one name.

The surface language follows code-as-plan visual programming [19, 57], but is deliberately restricted to the operations in table 4. For claim c_i, the planner writes `check(ci, <expr>)` with one operation or a conjunction of operations. At this stage the planner has no video input: the line states what evidence would settle the claim, not whether the claim is true.

Bounded recovery. The assembled surface plan is validated as a whole because its important constraints are cross-references: every check must consume existing actions, every identifier must denote a shared entity or event, and every claim must have a check. A failing group may be asked again with its own parser or type complaints and a restatement of the format, but never with a suggested answer. After a fixed number of attempts, an unusable claim falls back to direct semantic verification. The fallback is recorded and charged as a model call; it is not counted as a specialist measurement. A failed naming pass sends all claims through the same charged fallback, so plan failure never makes a prompt appear cheaper by silently dropping claims.

4.4 Typed plan representation and static validation

The compiler turns the surface program into a typed directed graph

$$G_\Pi = (\mathcal{A}, \mathcal{Q}, \mathcal{E}, b), \qquad a_j = (o_j, \alpha_j, \omega_j, r_j), \qquad b : \mathcal{Q} \to \{1, \dots, N_p\}. \tag{4}$$

Here $\mathcal{A}$ contains evidence-producing actions, including general semantic passes and specialist calls; $\mathcal{Q}$ contains typed checks; $\mathcal{E}$ points from a prerequisite to its consumer; and $b(q) = i$ binds check q to claim c_i. An action records its operation o_j, arguments α_j, declared scope policy ω_j, and result type r_j.

Before any action runs, the graph must satisfy

$$\mathrm{Valid}(G_\Pi) = \mathrm{Parse}(G_\Pi) \wedge \mathrm{Refs}(G_\Pi) \wedge \mathrm{Typed}(G_\Pi) \wedge \mathrm{Acyclic}(G_\Pi) \wedge [\forall i,\ \exists q \in \mathcal{Q} : b(q) = i]\,. \tag{5}$$

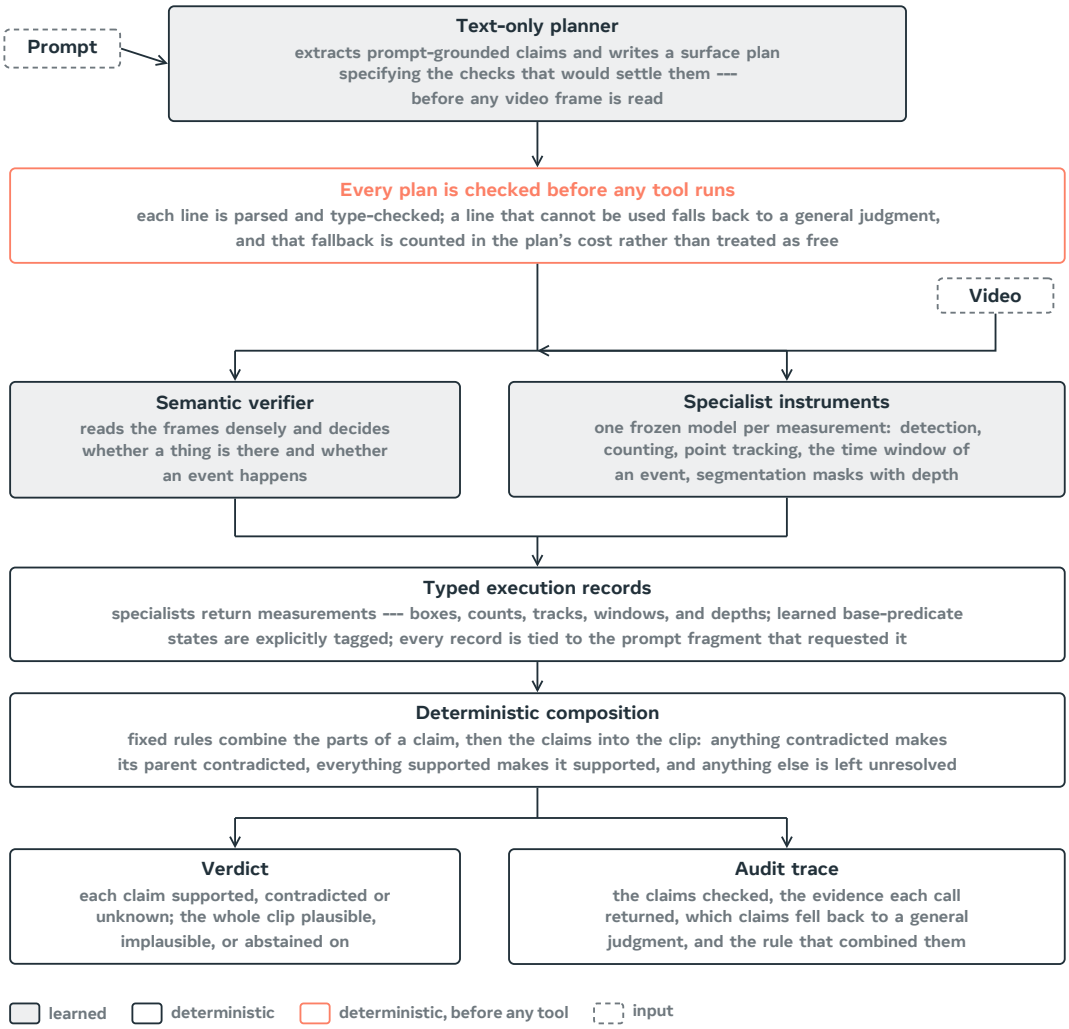


Figure 11 `PhyReAct` dataflow. The text-only planner receives the prompt and writes claims and a surface plan; the compiler expands and validates it before the video reaches execution. The video-aware verifier emits explicitly learned base-predicate states, whereas specialists emit typed measurements. Only the final claim and clip aggregation is wholly deterministic. Shaded boxes are learned components; outlined boxes are deterministic.

Surface operation	Question posed	Typed check
`judge("assertion")`	dense-frame semantic verification of the complete assertion	general semantic
`exists("object")`	whether the object is visible at any decoded time	existence
`eq(count("phrase"), N)`	whether measured cardinality equals N	count
`before(window(A), window(B))`	whether event A precedes event B	temporal order
`during(window(A), window(B))`	whether A is temporally contained in B	temporal containment
`rel("A||B", relation)`	a fixed-vocabulary spatial relation	spatial
`traj("phrase", relation, target)`	a fixed-vocabulary motion predicate	trajectory
`sound("description")`	whether the audio track contains the event	audio occurrence
`text("STRING", carrier=...)`	whether the named surface renders the string	rendered text

Table 4 The planner's surface vocabulary. `count` and `window` are structural terms. The deterministic compiler expands a surface operation into its required execution closure. Thus `before` adds occurrence gates and two timing actions; `rel` adds co-visibility and mask actions and, for *behind* or *in front of*, depth actions. These dependencies need not be written as extra surface operations, but they must be present in the compiled graph. A sound operation reads the audio track; no audio track yields **unknown**, not a negative measurement.

Refs checks identifiers and dependency closure; Typed checks operator arity, fixed relation vocabularies, and whether every check receives the result types it requires. For example, a directed trajectory must carry a target, and a depth-ordering check must consume both masks and depth. An invalid whole line falls back to the general verifier; an invalid term can be replaced by a recorded generalist sub-check. In both cases the substitution is charged and reported rather than guessed.

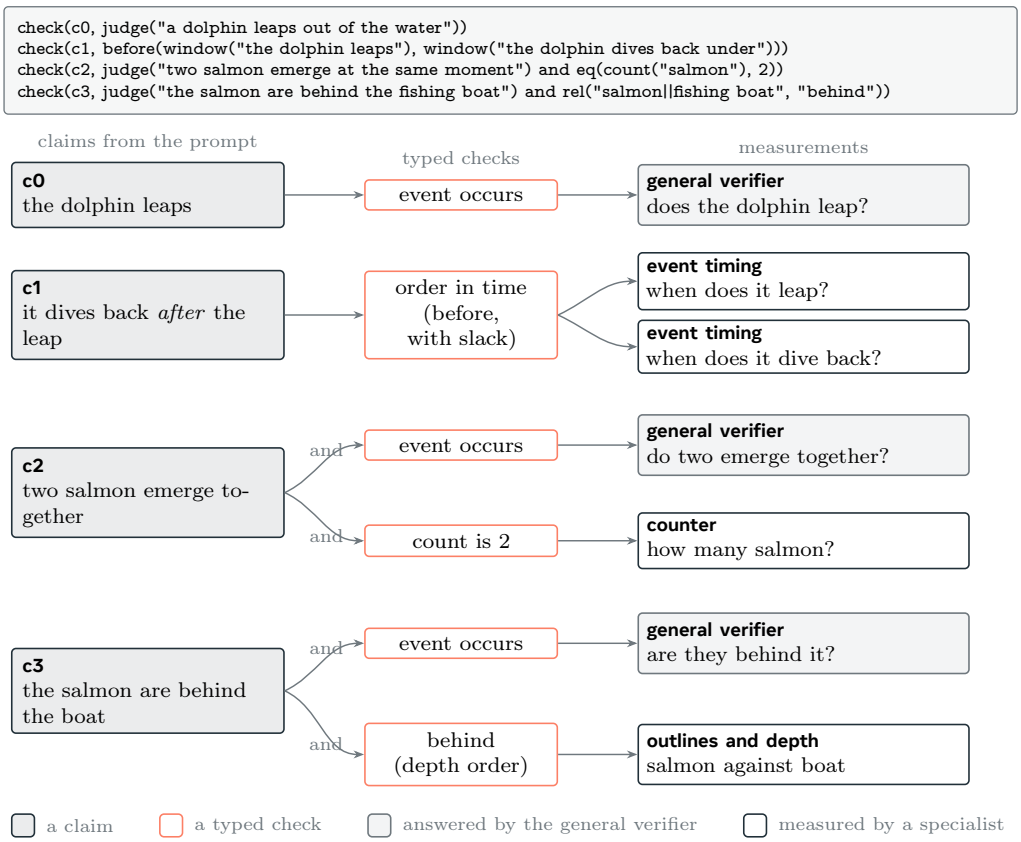


Figure 12 A surface plan after compilation into claims, typed checks, and evidence-producing actions. A numeral adds a count check beside a semantic assertion; an ordering consumes two event windows after occurrence gating; and a depth relation consumes localization, masks, and depth supplied by the compiler's dependency closure. The graph is illustrative of the planner's vocabulary rather than an additional learned verdict.

4.5 Timeline-conditioned multi-tool execution

Execution follows a topological order. Each action's compiled scope function g_j converts prerequisite records into either the full frame index set or a declared local subset, after which the appropriate backend is called:

$$\Omega_j = g_j(\{R_k : (a_k, a_j) \in \mathcal{E}\}, \mathcal{T}_\mathcal{V}), \qquad R_j = F_{o_j}\big(\mathcal{V}|_{\Omega_j}, \alpha_j\big). \tag{6}$$

The scope function is part of the validated program. A usable measured window is padded, given a minimum duration, and converted to the explicit indices whose timestamps fall inside it. A missing, invalid, or overly broad window falls back to the whole clip. This conservative fallback avoids interpreting failure to localize as evidence that the requested event or relation is absent.

Occurrence gates timing. A returned time window is not evidence that the queried event occurred: a forced localizer may return its best candidate even for an absent event. Visible events are therefore confirmed first by the dense-frame semantic verifier, while audible events are confirmed on the audio track. Only supported occurrences allow their intervals to enter a temporal predicate. For a relation $R \in \{\text{before}, \text{during}\}$,

$$z_{R(A,B)} = \begin{cases} \mathsf{C}, & \mathsf{C} \in \{o_A, o_B\}, \\ \mathsf{U}, & \mathsf{U} \in \{o_A, o_B\} \text{ or } W_A \text{ or } W_B \text{ is unavailable}, \\ \rho_R(W_A, W_B), & o_A = o_B = \mathsf{S}, \end{cases} \tag{7}$$

where o_A, o_B are occurrence states and W_A, W_B are measured windows. Thus timing may compose an order but can never establish occurrence.

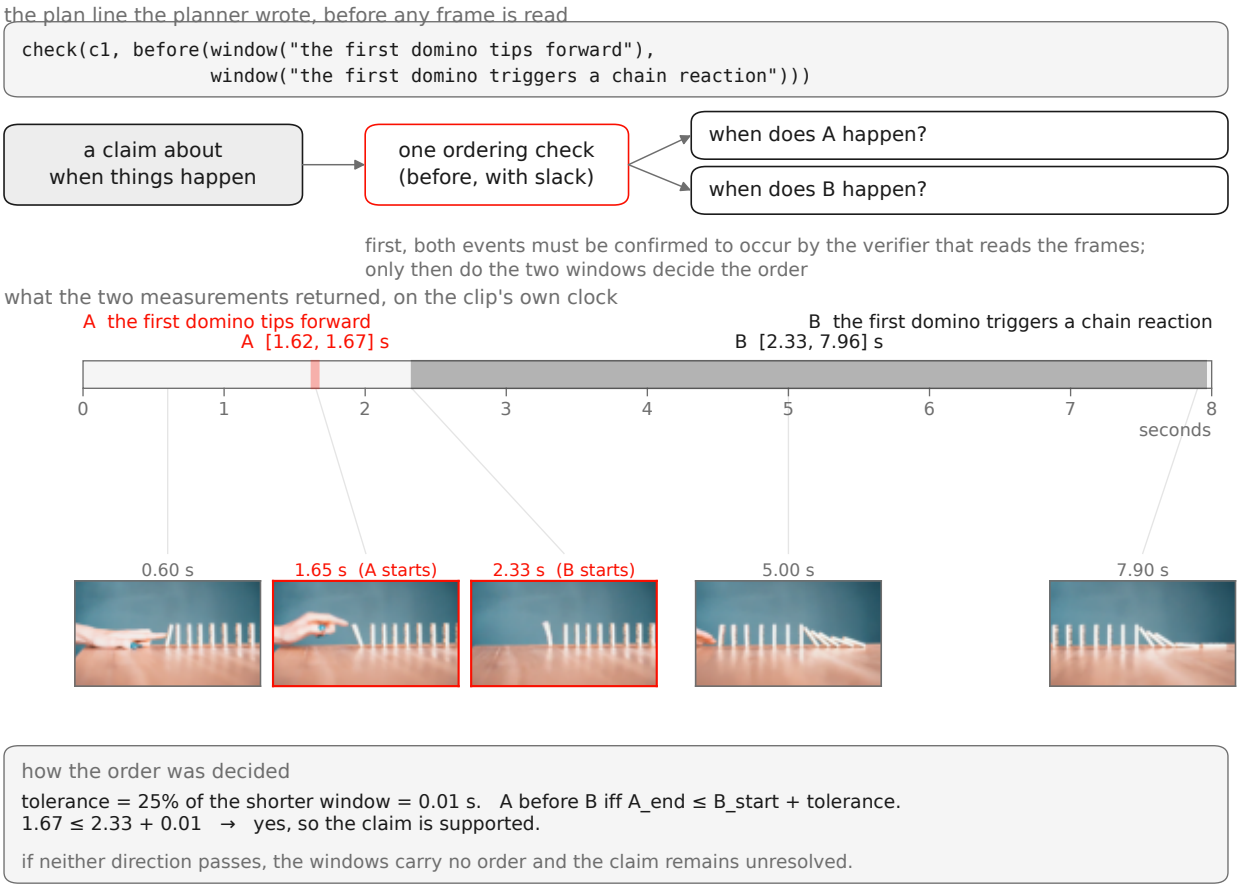


Figure 13 A real temporal trace. The planner writes the line before reading any frame. The verifier first confirms both events, the timing actions then return $W_A = [1.62, 1.67]$ s and $W_B = [2.33, 7.96]$ s, and equation (8) resolves their order. Every number and displayed frame comes from the recorded run.

Temporal arithmetic. For positive-duration windows $W_X = [s_X, e_X]$, the recorded executor uses a boundary slack equal to one quarter of the shorter window:

$$\delta(A, B) = \tfrac{1}{4} \min(e_A - s_A, e_B - s_B),$$
$$\rho_{\text{before}}(W_A, W_B) = \begin{cases} \mathsf{S}, & e_A \le s_B + \delta(A, B), \\ \mathsf{C}, & e_B \le s_A + \delta(A, B), \\ \mathsf{U}, & \text{otherwise.} \end{cases} \tag{8}$$

The slack absorbs small endpoint uncertainty, so it can tolerate a boundary overlap no larger than δ; if neither direction passes, the intervals do not resolve an order and the state is U. With the quarter-window rule and positive durations, the first two cases cannot both hold. Containment is directional and uses the same boundary slack: clear containment supports, clear reverse containment contradicts, and boundary-indistinguishable cases remain unknown. These predicates are a tolerance-scaled subset of Allen's interval algebra [1].

Exact-request reuse. Two actions share one execution only when their complete realized signatures match:

$$\chi_{\mathcal{V}}(a_j) = (\mathrm{id}(\mathcal{V}), o_j, \alpha_j, r_j, \Omega_j), \qquad a_j \sim a_k \iff \chi_{\mathcal{V}}(a_j) = \chi_{\mathcal{V}}(a_k). \tag{9}$$

The executor calls each equivalence class once, following exact common-subexpression reuse [49, 51]. Because the realized scope Ω_j is part of the signature, whole-clip and localized requests never merge merely because their natural-language queries are similar.

Capability	Running backend	Returned object
Semantic predicate	dense timestamped Qwen3-VL pass [48]	learned base state with localized rationale
Object localization	LLMDet open-vocabulary detector [15]	box proposals, not an existence verdict
Object count	CountGD-Box [2, 3] (SAM 2.1 internal)	cardinality of a named set
Kinematics	TAPNext-style point tracking [78] and kinematic predicates	timestamped positions and motion features
Event timing	Qwen3-VL over timestamped frames; optional dedicated grounder [75]	interval, absence, or refusal
Spatial relation	SAM 3 masks [8], monocular depth, and localized reasoning	masks, depth, and a tagged relation state
Rendered text	optical character recognition on localized frames	literal glyph transcription
Audio	FlexSED sound-event detection [20]	sound intervals on the audio track

Table 5 Execution backends and their outputs. Localization proposals seed later measurements but do not prove existence. Specialists provide typed measurements; learned semantic decisions are identified as such. No holistic video-quality model is used to produce the final clip decision.

Execution backends. Table 5 lists the backend that actually runs, rather than a stronger backend that an adapter may only declare. There is no holistic scorer in the verdict path. Three sub-checks remain explicitly learned because they resist full symbolization: subtype identity, dense action-event recall, and depth-ordered spatial interpretation on localized evidence. Their outputs are tagged as learned base-predicate states; they are not presented as deterministic consequences of a raw measurement.

4.6 Evidence records and provenance

Each action is serialized as a claim-bound record

$$R_{j,i} = \big(\mathrm{id}(\mathcal{V}), p, s_i, o_j, \Omega_j, \sigma_j, \mu_j, h_j, \kappa_j\big), \qquad \sigma_j \in \{\texttt{measured}, \texttt{abstained}, \texttt{errored}\}. \tag{10}$$

The payload μ_j contains either a typed measurement or an explicitly tagged learned base state; h_j contains the native-output and artifact hashes; and κ_j contains calls and accelerator time, with optional wall time, price, and token counts. Shared execution may bind the same record to several claims, but its cost is charged once by execution identifier.

The three statuses are mutually exclusive. A `measured` record carries a payload and no abstention or error; `abstained` carries a reason and no payload; `errored` carries an infrastructure message. A valid negative measurement—for example a measured count of zero or a complete usable track that does not exhibit the requested motion—remains `measured` and may contradict a claim. Failure to track because of blur, occlusion, low quality, or an instrument refusal is `abstained`, not a negative observation. A terminal `errored` record is retried and, if still terminal, produces no semantic verdict; the clip is marked $\perp_{\mathrm{infra}}$ and excluded from the completed evaluation set rather than relabeled **unknown**.

Specialist schemas reject judgment-shaped fields, nonfinite numbers, duplicate keys, and noncanonical payloads. Every record carries the full prompt and the verbatim source span s_i, and heavy artifacts are content-addressed by path and byte hash. The guarantee is intentionally narrow: a specialist cannot emit the final prompt-satisfaction label, and accepted evidence is attributable and replayable. The schema cannot make a learned measurement unbiased or detect a confidently wrong but well-formed payload.

4.7 Three-valued verification and aggregation

For a completed trace, every sub-check resolves in

$$\mathbb{V} = \{\mathsf{S}, \mathsf{C}, \mathsf{U}\},$$

standing for **supported**, **contradicted**, and **unknown**. A typed resolver ψ_q maps measured parent payloads to a sub-check state; it is deterministic for count, timing, track, transcription, and other explicit predicates, and is learned only for the semantic sub-checks named above:

$$z_q = \begin{cases} \psi_q(\{\mu_j : a_j \in \mathrm{Pa}(q)\}), & \sigma_j = \texttt{measured} \text{ for all required } a_j, \\ \mathsf{U}, & \text{a required action abstained or produced no usable evidence,} \\ \perp_{\mathrm{infra}}, & \text{a required action has a terminal infrastructure error.} \end{cases} \tag{11}$$

Learned semantic checks use an asymmetric gate: they may contradict only when a positively contradicting feature is visible; unresolved visual evidence, unparseable output, or missing modality yields U. This avoids turning failure of an instrument into an accusation about the video.

Negation is applied to the positive sub-check state, never by asking a tool to measure a negative quantity:

$$\neg_3\mathsf{S} = \mathsf{C}, \qquad \neg_3\mathsf{C} = \mathsf{S}, \qquad \neg_3\mathsf{U} = \mathsf{U}. \tag{12}$$

Let $K_i = \{q \in \mathcal{Q} : b(q) = i\}$ be the validated checks for claim c_i, after applying equation (12) where required. Claim and clip composition are

$$y_i = \begin{cases} \mathsf{C}, & \exists q \in K_i : z_q = \mathsf{C}, \\ \mathsf{S}, & \forall q \in K_i : z_q = \mathsf{S}, \\ \mathsf{U}, & \text{otherwise,} \end{cases} \qquad Y = \begin{cases} \perp_{\mathrm{infra}}, & \text{the trace terminates with an infrastructure error,} \\ \textsc{Implausible}, & \exists i : y_i = \mathsf{C}, \\ \textsc{Plausible}, & N_p \geq 1 \ \wedge \ \forall i : y_i = \mathsf{S}, \\ \textsc{Abstain}, & \text{otherwise.} \end{cases} \tag{13}$$

The order of the cases matters. One contradiction is sufficient for an implausible clip; plausible is emitted only when every planned claim is supported; and a trace with no contradiction but at least one unknown abstains. Plausible therefore summarizes the validated claims that were executed, rather than certifying every possible physical property of the clip.

4.8 Diagnostic interface: the contradiction packet

The useful refinement output of a `PhyReAct` run is not only the scalar clip state Y, but the localized contradiction packet

$$\mathcal{F}(p, \mathcal{V}) = \left\{ \left(c_i, \{R_{j,i}\}_j, \Omega_i, B_i\right) : \ y_i = \mathsf{C} \right\}, \tag{14}$$

where Ω_i is the union of available temporal support and B_i contains available box tracks or masks. Because the packet is assembled from the same auditable trace that produced the verdict, it retains full provenance: each contradicted claim carries the evidence records that support the conclusion and the region of the clip where the failure occurs. The packet therefore states *what* failed, *on what evidence*, and *where*. This packet is what the framework of Section 6 turns into an outward control signal: its space–time localization is exactly what lets a contradiction be mapped onto a local edit of the computed generator control (Section 6.4, (22)), closing an independent-critic→generator loop.

Closed-loop refinement is not run in this report. The critic and the simulation-conditioned generator are evaluated on disjoint data, and no `PhyReAct` packet has yet been used for preference optimization, denoising guidance, candidate selection, or localized regeneration. Equation (14) defines the interface only; we make no claim that the current generator has been optimized by the critic or that any packet has made a generated clip more physically correct.

5 Evaluation Protocol and Measurement

This section states how the critic is measured and reports what those measurements returned. A dash marks a measurement that has not been run.

Implementation. Qwen3-VL-30B-A3B-Instruct [48], served in BF16 without quantization and tensor-parallel over two GPUs, is used for both the text-only planner and the video-aware semantic verifier. These are separate calls: the planner receives the prompt and claim state but no frames, while the verifier receives densely decoded, timestamped frames and a single declared predicate. The specialist revisions are frozen for the evaluated run and the running backends are those in table 5.

Target and ground truth. The evaluation core of section 3, scored under the conventions of section 3.1. Human labels are the only ground truth.

Matching is outside the critic. To decide whether a critic output corresponds to a human-flagged flaw, a separate protocol uses two judge models (a GPT-family and a Gemini-family model, majority over runs). These judges perform *flaw matching only*; they are not components of the critic, do not produce its verdict, and are not treated as ground truth. Any "majority-of-three" in this project refers to such a model-vote protocol, never to human annotators.

Dense-frame requirement. The critic must observe the video densely. The evaluated components extract frames densely under a logged sampling policy, and any measurement obtained from a sparsely sampled input is excluded. A re-run counts only if it uses the dense path on the unchanged frozen target with BF16 tools at pinned revisions.

The comparison. The critic is compared against per-claim verification: the same claims, verified one at a time by the general verifier, with the question phrased as the critic phrases it, since the phrasing alone changes how readily a verifier answers. Because a detector call, a tracker call and a full vision-language judgement do not cost the same, cost is reported as model and tool calls per clip. Uncertainty is estimated by resampling whole clips, because flaws within a clip are not independent.

Planner validity and measurement coverage. Before end-to-end evaluation, the prompt-only planning stage was inspected without video or tool access. On 233 claims from 22 annotated clips, every claim produced a valid, compilable plan at one to two planner calls per clip. Twenty-seven claims (11.6%) contained at least one specialist action that survived validation, compared with 2.3% for an earlier tabular plan format. On deliberately measurement-dense diagnostics the corresponding rates were 19/30 for counting, spatial, and ordering claims, and 10/10 for synthetic counting prompts. These diagnostics test routing and syntax; they are not estimates of verdict accuracy or coverage on the evaluation core. The present surface vocabulary also lacks an "on top of" spatial predicate and rejects negated quantitative measurements; such terms become recorded, charged generalist fallbacks.

5.1 Results

Table 6 reports the paired comparison completed to date: the plan-based critic against per-claim verification, on the 149 clips of the evaluation core both systems have scored. Both receive the same claim texts and the same matching protocol, and the per-claim baseline is wording-controlled—asked its questions in the form the plan-based critic uses, because the phrasing alone changes how readily a verifier answers. Scoring follows section 3.1.

The plan-based critic catches **18 more flaws**, 6.3 points of recall, for about one extra call per clip. Resampling whole clips with a fixed seed puts that difference at 95% confidence in $[+8, +28]$, so it is a real difference on this set rather than sampling noise.

Two qualifications bound what the number means. It is a trade of accuracy against cost, not a dominance result: the plan-based path spends about 7% more calls per clip, and no run holds the two systems to an equal call budget. And this set is the set the system was developed against, so the figure describes fit to it rather than generalization.

System	Flaws caught of 286	Before collapsing duplicates, of 304	Allegations that landed	Calls per clip
Plan-based critic	**197 (68.9%)**	197 (64.8%)	40.1%	13.0
Per-claim verification, one claim at a time	179 (62.6%)	179 (58.9%)	40.2%	12.1

Table 6 Paired results on the 149 clips of the evaluation core that both systems completed; the one missing clip is an infrastructure loss. Those clips carry 304 of the core's 306 flaw entries, which collapse to 286 distinct spans (section 3.1); the second column repeats the same allegations against the uncollapsed flaw list. "Allegations that landed" is the share of each system's accusations that the matching protocol linked to a human flaw; it is reported because recall alone rewards over-flagging. Calls count model and tool invocations and exclude the matching protocol, which sits outside the critic.

Where the misses come from. Of the 89 flaws the plan-based critic missed, two were never decomposed—no claim in the plan covered them. Eight were flagged but not linked to the human flaw by the matching protocol. The remaining seventy-nine were checked and cleared: a claim covered the flaw and the critic judged the video fine, and seventy-six of those were decided by a yes-or-no judgement with no measurement attached rather than by a specialist returning a wrong number.

The bottleneck is therefore measurement coverage rather than decomposition or specialist accuracy. Of 1,917 executed checks, 1,414 ask whether an event occurs and 233 whether something exists, against 22 spatial relations, 19 trajectories, 18 orderings, 18 text reads and 16 counts: 14.1% are measurements. The claims this data produces rarely ask the plan to measure anything, which makes claim typing, not tool accuracy, the next lever.

Cost. Per-clip wall time over the 149 clips has a median of 102 s and a mean of 373 s; the ninetieth percentile is 1,087 s and the maximum 6,119 s. The mean sits far above the median because a minority of clips invoke dense per-frame specialists, and those clips set the wall-clock rather than the typical one.

What this does not cover. The paired set is 149 of the 150 benchmark clips. The cost-matched comparison in both directions and per-clip GPU-second accounting remain unrun, as does every flaw-level external comparison in table 8; the rating-level comparison of section 5.2 is separate and has been run.

5.2 Agreement with human ratings, against a purpose-trained judge

Flaw recall is not commensurate with what most published evaluators emit, which is a clip-level scalar rather than an allegation that can be matched to a localized human flaw. There is, however, common ground: both kinds of system can be asked to rate a clip on the scale a human used. VideoScore2 [22] is the natural opponent there—a judge trained for exactly this task, with a published evaluation script.

The comparison follows that script rather than a protocol of our own. On a sample of VideoPhy-2 [6] test clips, each judge emits an integer 1–5 per dimension; agreement with the human rating is scored as exact accuracy, accuracy within one point, and the linear and rank correlations (PLCC and SRCC), with unparsed predictions excluded pairwise. The sample is 200 clips drawn uniformly with a fixed seed, and the sample, the score definition and the metrics were registered before any result existed; nothing in the scoring function is fitted to these clips. Of the 200, 55 carry the benchmark's *hard* flag.

Table 7 reports both judges, and the two families of metric disagree: neither judge leads on both. VideoScore2 is better calibrated to the human's absolute point on the scale, winning exact agreement (33.0% against 31.0%) and agreement within one point (83.0% against 76.0%). The critic correlates better with the human ordering. Bootstrapping the paired per-clip difference over the same 200 clips, the rank-correlation advantage is $+0.138$ with a 95% interval of $[+0.005, +0.277]$, which excludes zero only narrowly; the linear-correlation advantage is $+0.110$ with an interval of $[-0.018, +0.242]$, which does *not* exclude zero. On the full sample, then, the ordering advantage is real but slight, and the linear one is not established.

	All 200 clips				Hard 55	
Judge	**exact**	±1	**PLCC**	**SRCC**	**PLCC**	**SRCC**
VideoScore2 [22]	**33.0%**	**83.0%**	0.353	0.327	0.239	0.151
This work	31.0%	76.0%	**0.464**	**0.465**	**0.480**	**0.460**

Table 7 Prompt match: agreement with the human semantic-adherence rating on 200 pre-registered VideoPhy-2 test clips, scored with VideoScore2's own evaluation script. VideoPhy-2 also carries a physical-commonsense rating; the critic emits no score on that scale, so that dimension is outside this comparison.

Evaluator	Output it produces	Caught	Recall	Calls
This work, plan-based	localized allegation	197	68.9%	13.0
Per-claim verification	localized allegation	179	62.6%	12.1
Question-decomposition scoring [10, 24]	per-question answers	—	—	—
Alignment scoring [23, 33]	clip-level scalar	—	—	—
Learned quality scoring [21, 22]	clip-level scalar	—	—	—
Modular video question answering [12, 43]	per-question answers	—	—	—

Table 8 The intended external comparison, stated before it is run. The first two rows are the measured systems of table 6, on its 149 clips and 286 flaws. Dashes are unrun. A clip-level scalar has no allegation to match against a localized human flaw, so scoring it requires a stated rule for turning a score into a decision; that rule will be fixed in advance and reported with the result.

The hard slice is where the two separate. On those 55 clips the critic's advantage is +0.242 linear ($[+0.018, +0.511]$) and +0.309 rank ($[+0.050, +0.583]$), both excluding zero, as VideoScore2's rank correlation falls to 0.151 while the critic's holds at 0.460. That is the pattern one would expect if evidence assembled per claim degrades more gracefully on difficult clips than a single learned scalar, though 55 clips is a small sample and the intervals are correspondingly wide.

Why only one of the two scales is reported. VideoPhy-2 rates each clip on a second scale, physical commonsense, and the registered plan mapped the critic's output into that column as well. That analysis is not reported as a result, because it does not measure what the column is for. The critic's score is the fraction of a clip's claims that its checks contradicted, converted to a 1–5 number; rating how physically plausible a video looks is a different output from deciding whether a stated claim is contradicted, and the critic emits no score on the former scale. Comparing the one against the other measures the mismatch, not the capability. Producing such a rating would need a head that consumes the kinematic and trajectory checks directly, which does not exist here. This analysis therefore provides no evidence about physical-commonsense rating, and none about physics-flaw detection either; the flaw-level evidence for that is table 6, on human-localized flaws.

Two further limits. This is one fixed sample of 200 clips, so it bounds agreement on that sample rather than across the benchmark. And the two judges are not cost-matched: the critic runs a decomposition and a set of tool calls per clip, VideoScore2 a single forward pass, so nothing here speaks to efficiency.

5.3 The intended flaw-level comparison

The rating comparison above does not settle the flaw-level question, because a clip-level scalar cannot be matched to a localized human flaw. Table 8 fixes that comparison, so that the protocol is stated before the numbers exist. Each will be scored on the same frozen set, against the same human labels, with the same matching protocol, and each will be given the same claim decomposition where it accepts one.

The set was inspected and used during optimization, the wording-controlled per-claim baseline was introduced after treatment outputs had been observed, and no retroactive held-out split exists within these 150 clips; a

claim of generalization therefore requires evaluation on a new frozen set.

6 Framework: Learning as State Optimization

The preceding sections described three concrete artifacts that already run: a physics-guided generator (Section 2), a 1,500-clip human-authored defect benchmark (Section 3), and a physically grounded, tool-using critic whose plan-based adjudication we measure in Section 5. This section steps back and states the single abstraction that ties them together and defines what `PhyReAct` is meant to become. Our thesis is that a physical-reasoning agent should keep everything it has learned not in its weights but in one *human-readable, typed, provenance-carrying state*, and should improve by *optimizing that state rather than by updating weights*. Every improvement is then an edit to an object a person can read, version, and veto, and the arc of the framework culminates in a system that evolves itself without ever touching a weight.

We are deliberate about the honesty boundary. Only three components rest on experiments (Sections 2, 3 and 5); everything else in this section is framework and roadmap. Because all learning is state optimization on a readable S, this boundary can be drawn cleanly *inside* the same formalism rather than hidden, and Table 9 states it plainly.

6.1 From ReAct and agent memory to a physical agent with typed state

Agentic reasoning in the ReAct lineage interleaves *reason*, *act*, and *observe* steps, letting a model call external tools and condition on their returns [17, 54, 71]. Program-of-thought variants push this further: the chain of thought is emitted *as a program* that composes tool calls [19, 28, 50, 53, 57, 64]. Orthogonally, a fast-growing agent-memory literature asks where an agent's accumulated experience should live—episodic transcripts, distilled semantic priors, or reusable procedures [7, 25, 29, 32, 39, 47, 65, 67, 76, 77]. `PhyReAct` sits at the intersection and makes one decisive change: it *physicalizes* the ReAct harness and upgrades the *carrier* of memory. Where prior agents store free-text notes or latent vectors, `PhyReAct` stores structured physical-verification traces with full provenance. This single choice is what makes the agent auditable by construction, and it is the throughline of the framework below.

6.2 The unifying abstraction

`PhyReAct` maintains a single, human-readable *agent state*

$$S \;=\; (\Pi,\; M,\; K), \tag{15}$$

and it improves by optimizing S. The underlying reasoning model and the low-level physical experts are frozen; all accumulated competence lives in S, which is inspectable, diffable, versionable, and revertible. "Training" and "inference" are the same readable procedure run in different directions: one map reads S to act, and channels write to S (or to the generator) to improve.

Π — **operator library (procedural memory).** A set of reusable, already-compiled measurement operators. Each operator $o \in \Pi$ is a closure mapping a video and typed arguments to a physical measurement, and it carries an explicit *cost node* $c(o)$ recording its spending (compute, expert calls, latency). Π grows monotonically as recurring reasoning sub-procedures are packaged into named operators.

M — **experience library (episodic + semantic memory).** Versioned, human-readable priors distilled from past traces: nearest-neighbor *plans* for similar prompts and *physics-failure priors* over common defect modes. M keeps *evidence* (what was measured) separate from *belief* (what we now expect): evidence is non-negotiable, belief is defeasible and can be overridden by fresh measurement, so no distilled prior is ever mistaken for an observation.

K — **external knowledge channel.** Retrieved external or social knowledge (physical facts, domain conventions, human corrections) admitted only through a candidate-and-pruning discipline. K is the deliberate *open* port that keeps the internal improvement loop from closing on itself (Section 6.6).

The rest of the framework is exactly two kinds of maps over this state: a single-episode *read* operator that consumes S and emits an auditable trace (Section 6.3), and *write* channels that update the generator and S

from that trace. The six design pillars of the paper are recovered as special cases of read-vs-write over S: P1/P2 and the read side of P4 are "read S to produce a trace"; P3 and P6 are "write S"; P5 is "write the generator"; and the supervisory side of P4 wraps the writes.

6.3 The single-episode operator (read path): physicalized ReAct as a program over Π

Given a prompt–video pair (p, V), one episode is a deterministic pipeline that only *reads* S:

$$\begin{aligned} \text{retrieve:} \quad & (\pi_p, m_p, k_p) = r(S, p), && (16)\\ \text{compile:} \quad & (C, G) = \text{Reason}(p, V \mid \pi_p, m_p, k_p), && (17)\\ \text{act–observe:} \quad & R = \text{Exec}_\Pi(G, V), && (18)\\ \text{adjudicate:} \quad & (Y, F) = \Phi(C, R), && (19)\\ \text{emit:} \quad & T = (C, G, R, Y, F). && (20) \end{aligned}$$

Here C is a set of *typed physical obligations*—explicit, machine-checkable statements of what the video must satisfy to honor the prompt (object presence and count, contact and support, trajectory continuity, depth ordering, on-screen text, audio events). G is a *program over* Π: reasoning is not free-form chain-of-thought but a composition of operator calls that the model writes and the runtime executes, in the tradition of program-of-thought and modular visual reasoning [19, 28, 57, 64]. Executing G drives the classic reason→act→observe loop of ReAct-style agents [71], but every `act` is a call into a *frozen* physical expert operator—open-vocabulary detection [15], counting [3], point tracking [78], segmentation [8], monocular depth, OCR, and audio event detection [20]—registered in Π. This is the sense in which `PhyReAct` *physicalizes* ReAct: it inherits the control-flow harness from mainstream tool-using agents [50, 53] and specializes it with explicit physical obligations and physical-measurement operators **(P1)**. This general form is instantiated concretely by the critic of Section 4; Equation (21) below is the same three-valued verdict used there.

Each operator returns a measurement together with a *gate* on its own scope: when an operator's preconditions are not met (out-of-scope object, untrackable occlusion, no audio track), it emits **abstain** rather than a fabricated value. The adjudicator Φ in (19) is a *deterministic* roll-up from evidence R to a three-valued verdict

$$Y \in \{\textbf{supported}, \textbf{contradicted}, \textbf{unknown}\}, \tag{21}$$

plus a *contradiction packet* F that localizes, for each unmet obligation, which measurement contradicts it and where. The three-valued codomain is load-bearing: **unknown** (and **abstain** for out-of-scope obligations) are first-class outcomes, so the critic can decline to certify rather than guess, and a **contradicted** verdict is never conflated with mere implausibility. The emitted trace $T = (C, G, R, Y, F)$ carries full provenance—every element of Y is back-traceable to the operator calls in G and the measurements in R.

CoT-as-program and cost nodes (P2). Because G is a program, recurring sub-procedures (e.g. "verify a bounce = track the object, detect the contact frame, check the pre/post vertical-velocity sign flip") can be packaged, named, and re-registered as new operators in Π. Each operator carries its cost node $c(o)$, so a program has a well-defined budget $c(G) = \sum_{o \in G} c(o)$ and the reasoner can trade breadth of measurement against spending. Over many episodes Π scales up into a growing procedural memory, and reasoning increasingly becomes *composition of trusted operators* rather than re-derivation from scratch. What is instantiated today is a fixed operator set Π_0 and a fixed compiler; the *scaling-up* of Π via distillation is roadmap (Section 6.8).

6.4 The outward channel: verification-guided refinement (P5)

The read path produces, for a failed clip, a localized contradiction packet F (defined in Section 4.8). Rather than merely report a score, the outward *control channel* maps F into a *local edit of the generator's control signal* and regenerates:

$$V' = \text{Gen}\big(p,\ \delta(F)\big), \qquad \delta : F \longmapsto \Delta(\text{control}), \tag{22}$$

where Gen is the physics-guided pipeline of Section 2 (MuJoCo simulation [58] driving a Wan 2.2 video model [59] through VACE control conditioning [27]), and $\delta(F)$ is a *targeted* edit to the simulation/control inputs addressing the specific violated obligation rather than resampling blindly. Because the control signal is

computed, not estimated, a contradiction localized in space and time maps naturally onto a local edit of the conditioning stream. What makes this channel trustworthy is that the critic is *independent and frozen*—not the generator grading its own output, in contrast to self-evaluating refinement loops [60] and verification-guided repair [11]. Both endpoints of (22) are built (Sections 2 and 5); the map δ and the closed loop are roadmap, and we claim no generator improvement from this channel yet.

6.5 Auditability by construction

Auditability here is not a logging layer added afterwards but a consequence of the formalism: because the learned object is the readable state S and every conclusion is a deterministic roll-up Φ over provenance-carrying measurements, any verdict Y can be replayed back to the operator calls in G and the measurements in R, and any state change $S \to S'$ is a human-readable diff of Π, M, or K. This is what later makes both weight-free self-evolution and its human supervision tractable: the reviewer inspects readable objects, not gradients.

We are explicit about what auditability does *not* claim. A **supported** verdict is a critic judgment, not ground truth—ground truth in this paper is human annotation. A deterministic roll-up removes randomness in the roll-up step but does not by itself resolve evaluator drift in the underlying operators or the compiler. A prompt that survives our checks is *plausible* with respect to the tested obligations, not a certificate of physical correctness. And a static critic improving does not entail that the generator has improved. These distinctions are enforced by the three-valued codomain (21) and by keeping evidence separate from belief in M.

6.6 The culmination: weight-free context-training self-evolution

The framework builds toward one capability: a critic that improves itself without ever updating a weight. Everything above—a readable state, a program over operators, provenance traces, an independent verdict—exists so that the system's own experience can be folded back into S safely. This inward *context-training* channel **(P3)** is the terminal form of `PhyReAct`.

Given a trace T and feedback ϕ (human labels, downstream outcomes, or a later, more complete measurement), we distill *candidate* state edits and accept them under a strict guard:

$$(\Delta\Pi, \Delta M, \Delta K) = \mathrm{Distill}(T, \phi), \tag{23}$$

$$S' = \mathrm{Accept}\big(S,\ \Delta\Pi, \Delta M, \Delta K\big). \tag{24}$$

Both successful and failed episodes are distilled (hindsight over traces [29, 47]): a success may add a plan to M or an operator to Π; a failure may add a physics-failure prior or a scope caveat. This is self-evolution that never touches weights, in the spirit of context-distillation and experience-memory agents [7, 25, 47, 54, 65, 76, 77], and it treats deployment-time adaptation as *learning as state optimization* rather than fine-tuning. The acceptance rule is where optimization becomes safe:

$$S' = \begin{cases} S \oplus \Delta, & \text{if } U_{\mathrm{val}}(S \oplus \Delta) > U_{\mathrm{val}}(S), \\ S, & \text{otherwise (do-nothing elitism),} \end{cases} \tag{25}$$

where U_{val} is a *held-out proxy* score and Δ ranges over distilled candidates. Candidates that do not strictly improve the proxy are pruned; when none does, the state is left unchanged. This do-nothing default gives a monotonicity guarantee on the proxy and is our first-order defense against context pollution.

Breaking the closed loop with external knowledge (P6). A system whose only inputs are its own traces—CoT → feedback → context → CoT—is a *closed* loop that can amplify its own biases: distilled priors condition future reasoning, whose traces distill into more priors. The external knowledge channel K is the deliberate opening. `PhyReAct` may issue retrieval or ask for external or social knowledge (physical facts, human corrections, domain conventions) and admit it into K under the same candidate-and-pruning discipline as (25), with an explicit *do-nothing* fallback: if no retrieved candidate improves the held-out proxy, nothing is admitted, and K-sourced facts are provenance-tagged distinctly from distilled beliefs so bad admissions remain revertible. Grounding the loop in outside information is thus not a license for uncontrolled ingestion; it is gated by the same elitism that protects S, so external knowledge can break the echo chamber without polluting it.

Pillar	Where it lives in the loop	Status
P1 Physicalized ReAct + operators	compile / act–observe over frozen experts [3, 8, 15, 20, 78]	Phase 0 (built)
P2 CoT-as-program + cost nodes	program G over Π, budget $c(G)$	Phase 0 partial
P4 Auditable trace + three-valued Y	trace $T = (C, G, R, Y, F)$, provenance	Phase 0 (artifacts)
P5 Verification-guided refinement	outward control channel δ, Gen	roadmap
P2$^+$ Operator-library scale-up	procedural-memory growth in Π	roadmap
P6 Breaking the loop	external channel K, candidate+prune	roadmap
P3 Context-training self-evolution	inward channel Distill, elitism (25)	roadmap (culmination)
P4$^+$ Human oversight	supervision + deployment gate	roadmap

Table 9 Each pillar is a projection of the same loop at a different radius. Only Phase 0 rows are backed by measurements; solid arrows in Figure 14 correspond to them, dashed arrows to the roadmap rows. No result in this paper is evidence for a roadmap row.

Because the learned object is a readable state, self-evolution stays honest about its own status: every prior distilled into M is a *critic-derived* prior, a summary of what `PhyReAct`'s own verification loop concluded, not verified truth. Retrieval quality, staleness of cached plans, and the propagation of a bad prior remain open problems. This entire inward channel—and the external opening—is roadmap: it is the destination the framework is designed to reach, and we neither run nor report it here.

6.7 Human oversight

A human is kept in the loop at a few specific gates rather than throughout: a reviewer may audit a trace T, veto or edit a proposed state update in (24), and approve whether a refined generation (22) or a new state S' is deployed. Human feedback also enters Distill as a first-class signal ϕ. Weight-free learning is what makes this lightweight: because updates are readable diffs of S, oversight is a matter of reading and approving, not retraining. The reviewing tooling and deployment protocol are roadmap; the artifacts they act on—traces, three-valued state, provenance—exist today.

6.8 What is built vs. what is roadmap

Because all learning is state optimization on a readable S, we can mark built-vs-roadmap directly on the formalism, and each pillar is a projection of the same loop at a different radius (Table 9). **Instantiated today (Phase 0):** the frozen generator pipeline Gen (Section 2); the 1,500-clip defect benchmark that exercises the read path (Section 3); the episode read operator (16)–(20) with a fixed operator set Π_0, typed obligations C, three-valued deterministic adjudication Φ, and provenance traces T; and the comparison in Section 5 of plan-based vs. per-claim adjudication ($197/286 = 68.9\%$ vs. $179/286 = 62.6\%$, recall-only, on the development core, *not* a held-out split). **Roadmap (Phase 1+):** the outward control map δ and the closed critic→generator loop (22); the scaling-up of Π; the culminating context-training self-evolution of M via (23)–(25); the external knowledge channel K; and human-in-the-loop deployment at scale. Every roadmap item is a *write* path in the same formalism whose *read* counterpart is already exercised, which is why we can state the vision precisely without implying it has been run. The executable plan follows in Section 7.

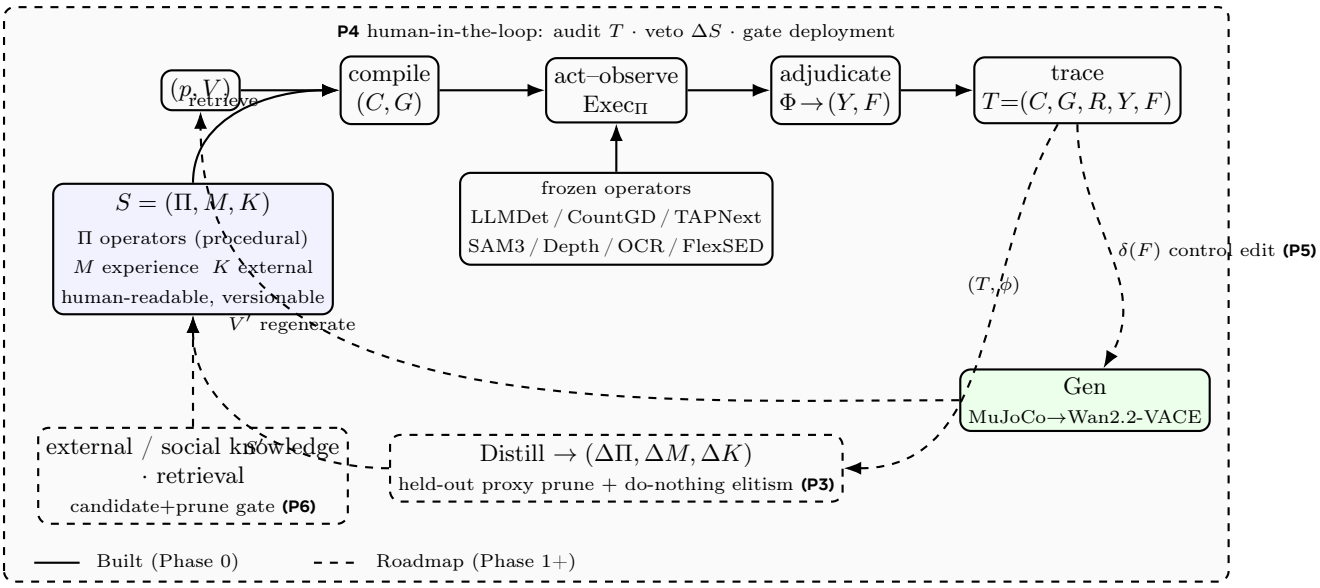


Figure 14 State-optimization view of `PhyReAct`. Everything the system learns lives in one human-readable state $S = (\Pi, M, K)$. A single *read* operator (top, solid = built) conditions on S, compiles (p, V) into typed obligations C and a program G over Π, runs frozen physical operators, and emits a provenance-carrying three-valued trace T. Two structurally dual *write* channels (dashed = roadmap), both driven by the same independent frozen critic, close the loop: the outward control channel edits the generator's control signal via $\delta(F)$ and regenerates (P5); the inward context-training channel distills T into candidate state edits accepted only under held-out proxy pruning with do-nothing elitism (P3). The external channel K breaks the internal loop (P6). A human-in-the-loop layer wraps both channels (P4). Solid arrows are instantiated in Sections 2, 3 and 5; dashed arrows are the roadmap of Section 7.

7 An Executable Roadmap

The framework of Section 6 defines one learned object, the state $S = (\Pi, M, K)$, and three maps over it: a read operator (built) and two write channels (roadmap). We phase the work accordingly. Phase 0 is the read path and the two frozen endpoints already instantiated in this paper; Phase 1 consolidates the real experiment onto an honest protocol; Phases 2–6 exercise the write paths, each with a measurable success metric and each a strict superset of what precedes it. No number below the Phase 0 line is a result; Phases 1+ state *targets and protocols*, not outcomes.

*Phase 0 — Built: the read path and frozen endpoints (*done*).*

- **Delivered.** The physics-guided generator Gen (MuJoCo → Wan2.2-VACE, Section 2); the 1500-clip human-authored defect benchmark (Section 3); the episode read operator with fixed Π_0, typed obligations C, three-valued deterministic adjudication Φ, and provenance traces T (Section 4); and the plan-based vs. per-claim comparison (197/286 = 68.9% vs. 179/286 = 62.6% recall on the development core, non-held-out, Section 5).
- **Covers.** P1 (physicalized ReAct read path), and the read side of P2 (programs over Π_0) and P4 (provenance traces).
- **Honest limits.** Recall-only, single non-held-out split; critic verdicts are judgments scored against human labels, not ground truth; a static evaluator result is not evidence any generator improved; the two write channels are not yet exercised.

Phase 1 — Held-out, precision+recall evaluation with provenance completeness.

- **Goal.** Promote the critic result from dev-core / recall-only to a held-out, precision+recall protocol, and guarantee every verdict carries a complete provenance trace T.

- **Needs.** A frozen held-out split of the benchmark disjoint from dev-core; a provenance schema for (C, G, R, Y, F).
- **Experiments.** Plan-based vs. per-claim adjudication on held-out clips with full confusion matrices; audit that every **contradicted** verdict localizes to frame-level evidence via F.
- **Success metric.** Held-out plan-based accuracy $\geq$ per-claim with non-degrading precision; complete provenance coverage.
- **Risk.** Held-out gap reveals dev-core overfitting; the three-valued codomain inflates **unknown** to dodge hard cases — mitigate by reporting the **unknown** rate alongside accuracy.

Phase 2 — Operator library with cost accounting (P2).

- **Goal.** Turn fixed Π_0 into a growing, versioned Π whose operators carry cost nodes $c(o)$ and whose programs have measurable budgets $c(G)$.
- **Needs.** An operator registry with provenance and versioning; a compiler that detects recurring G-sub-programs and promotes them into named operators; per-operator cost instrumentation wired into Φ.
- **Experiments.** Track the coverage/cost frontier as $|\Pi|$ grows; ablate frozen Π_0 vs. promoted Π at matched budget.
- **Success metric.** A Pareto improvement in obligation coverage per unit cost over the Phase 0 fixed pipeline at equal or lower $c(G)$.
- **Risk.** Library bloat and stale operators; promoted operators encode clip-specific shortcuts — mitigate with usage-frequency thresholds and held-out re-validation before promotion.

Phase 3 — Context-training self-evolution of M (P3).

- **Goal.** Distill traces (success and failure) into human-readable, versionable priors in M — nearest-neighbor plans and physics-failure priors — accepted under held-out proxy pruning with do-nothing elitism (25), keeping evidence separate from belief.
- **Needs.** A distillation function Distill; a held-out validation split U_{val} (new data, disjoint from the Phase 0/1 core); a diffable memory store.
- **Experiments.** Compare {no memory, retrieval-only, distilled M with elitism, distilled M without elitism} on the held-out split; a pollution stress test injecting wrong priors and checking elitism rejects them.
- **Success metric.** Monotone non-decreasing held-out proxy across accepted updates, a positive gap of "M with elitism" over both baselines, and zero acceptance of injected harmful priors.
- **Risk.** Belief leaking into evidence, or proxy overfitting; mitigate with the evidence/belief split and periodic held-out refresh.

Phase 4 — Closing the control channel critic $\to$ generator (P5).

- **Goal.** Implement $\delta : F \mapsto \Delta(\text{control})$ of (22), converting a contradiction packet into a *targeted* local edit of the MuJoCo/VACE control signal, and regenerate.
- **Needs.** A parameterization of Wan2.2-VACE control inputs by obligation type; a differentiable-or-heuristic edit policy δ; a re-adjudication harness.
- **Experiments.** On clips with **contradicted** verdicts, compare {blind resampling, prompt-only edit, targeted $\delta(F)$ edit}; report the fraction of previously violated obligations that flip to **supported** after regeneration, *human-verified*, and re-run the full obligation set C post-edit to report net defect change.
- **Success metric.** Targeted $\delta(F)$ regeneration repairs a significantly higher fraction of localized violations than blind resampling at equal generation budget, confirmed by human annotation (not by the critic alone), with no rise in unrelated-obligation contradictions.
- **Risk.** Goodharting the critic or whack-a-mole edits that fix one obligation while breaking another; mitigated by human verification and the net-defect audit.

Phase 5 — Breaking the loop with external knowledge K (P6).

- **Goal.** Open the internal CoT→feedback→context loop by admitting external/social knowledge and retrieval into K under the same candidate-and-pruning discipline, with a do-nothing fallback.
- **Needs.** A retrieval/ingestion interface; a candidate gate scored by U_{val}; provenance tagging that marks K-sourced facts distinctly from distilled beliefs.
- **Experiments.** On a bias-probe subset (defect modes under-represented in the internal traces), compare closed-loop ($K = \emptyset$) vs. open-loop (K admitted with gating); a noise-injection test on retrieved sources.
- **Success metric.** Reduced error on the bias-probe subset with no regression on the general held-out split; zero admissions when no candidate improves the proxy (fallback correctness).
- **Risk.** Context pollution from noisy or off-distribution sources; mitigated by the same elitism gate as S, plus source provenance so bad admissions are revertible.

Phase 6 — Human-in-the-loop deployment at scale (P4).

- **Goal.** Operate both channels under human supervision: reviewers audit traces T, veto/edit candidate state updates, and gate deployment of refined generations and new states S'.
- **Needs.** Trace-replay and state-diff review tooling; an approval workflow feeding Distill and K; audit logs over $S \to S'$ with rollback.
- **Experiments.** Held-out study measuring reviewer time per audited trace, inter-annotator agreement on state-update acceptance and on three-valued verdicts, and the catch-rate of injected bad updates that elitism alone would have missed.
- **Success metric.** Auditing cost per trace low enough to be practical, with human review reliably catching a measurable residual of harmful updates beyond the automatic gate.
- **Risk.** Reviewer fatigue and rubber-stamping (which would reintroduce an unaudited closed loop); mitigate by surfacing only elitism-borderline updates for human attention.

Ordering rationale. Phase 1 makes the real result honest; Phases 2–3 grow the two components of S that the inward channel writes (Π, M); Phase 4 exercises the outward channel; Phase 5 opens K; Phase 6 wraps everything in human oversight. Each phase reuses the Phase 0 read path unchanged, so at every step the system remains weight-free and auditable by construction.

8 Related Work

8.1 Physics-aware video generation and refinement

Controllable diffusion separates appearance synthesis from structural guidance. ControlNet introduced trainable branches for conditions such as depth and segmentation [73], VideoComposer extended compositional control to space and time [61], and Wan and VACE provide a scalable video backbone and a unified interface for reference, mask, and structural conditions [27, 59]. These methods determine how a condition enters a generator, but not whether that condition or the rendered pixels obey physics.

A complementary line derives guidance from physical reasoning or simulation. MotionCraft warps diffusion latents with simulated optical flow, while PhysGen estimates scene properties, performs rigid-body simulation, and refines the result with a video prior [35, 44]. DiffPhy injects language-model-inferred physical context through supervised fine-tuning [72]; PhyT2V iteratively rewrites prompts from physical-rule and mismatch reasoning [66]; and VLIPP turns vision-language reasoning into a coarse motion plan [69]. PhyCo learns continuous controls for physical properties from simulation data [46]. More tightly coupled systems place a simulator inside generation: PSIVG reconstructs a generated template, simulates object trajectories, and feeds them back at test time [14], while MoReGen coordinates language agents, simulation, and rendering for code-based synthesis [4].

Evaluation signals have also entered the generation loop. Binary VLM feedback has been used for preference optimization of object interactions [16]; Video-T1 searches candidate denoising trajectories with test-time verifiers [34]; and VideoRepair converts detected semantic misalignment into localized regeneration [30]. Our

generation testbed is deliberately narrower: known MuJoCo state is rendered into metric depth or silhouettes for a frozen Wan 2.2-VACE model. This avoids inverse reconstruction and generator training, but a physically coherent control signal does not certify that the synthesized pixels preserve its physics. The present work therefore evaluates the generator and critic separately; using `PhyReAct` evidence to guide generation remains future work.

8.2 Generated-video evaluation and physical benchmarks

General-purpose suites measure complementary aspects of generated video. VBench, T2V-CompBench, FETV, and EvalCrafter aggregate perceptual quality, temporal consistency, and prompt-alignment criteria [26, 36, 37, 56], while VideoScore and VideoScore2 learn to predict fine-grained human judgments [21, 22]. Prompt decomposition is also established: TIFA and the Davidsonian Scene Graph turn text into questions or propositions [10, 24], and VQAScore evaluates a text condition with an image-to-text model [33]. These approaches are valuable for ranking systems, but their final scores do not by themselves expose a measurement trace for a specific physical allegation.

Physics-specific evaluation has rapidly broadened. VideoPhy and VideoPhy-2 assess semantic adherence and physical commonsense, with VideoPhy-2 additionally labeling candidate physical rules as followed, violated, or undetermined [5, 6]. PhyGenBench spans common physical phenomena [41], Physics-IQ tests whether generative models encode physical principles [45], and PhyCoBench compares observed motion with optical-flow-guided future-frame prediction [9]. PhyWorldBench extends coverage to fundamental, composite, and deliberately anti-physical prompts, with prompt-specific evaluation standards [18]. These benchmarks provide essential generator-level comparisons and increasingly structured criteria. Our question is complementary: given an arbitrary prompt and clip, can an evaluator expose which physical obligation failed, what instrument measured it, when the supporting evidence occurred, and why insufficient evidence produced an abstention rather than a pass?

8.3 Agentic tool use and neuro-symbolic video reasoning

ReAct established the reasoning–action–observation loop, in which tool results can change the next reasoning step [71]. Related work explores how such behavior is acquired and organized. Toolformer learns when and how to insert API calls through self-supervision, while GPT4Tools transfers multimodal tool use to open language models through self-instruction [50, 68]. HuggingGPT uses an LLM controller to plan and route subtasks across expert models [53], while ControlLLM and ToolChain* search tool-call graphs or action trees [38, 79]. ReWOO and LLMCompiler instead plan before execution and make tool dependencies explicit [28, 64]. Feedback-based agents close another loop: Reflexion stores linguistic feedback in episodic memory across trials, whereas CRITIC invokes external tools to validate and iteratively revise an answer [17, 54].

Multimodal systems specialize these ideas to perception. Visual programs such as VisProg, ViperGPT, and CodeVQA compose vision modules through generated programs [19, 55, 57], while ProViQ, MoReVQA, the VideoAgent systems, and VideoTree adapt modular or selective observation to video [12, 13, 43, 62, 63]. Recent video agents make observation itself adaptive: CAViAR invokes modules and uses a critic to select reasoning traces, and LensWalk controls temporal scope and sampling density [31, 42]. Their primary setting is video question answering rather than physical verification of generated content.

Three systems are particularly close. VideoGen-Eval combines LLM prompt structuring, MLLM judgments, and temporal patch tools for multidimensional generated-video evaluation [70]. NeuS-V translates prompts into temporal-logic specifications, maps frame-level proposition confidence to a video automaton, and uses probabilistic model checking for alignment [52]. NeuS-E then identifies a weak proposition and failure frame and edits or regenerates the affected suffix [11]. Thus, agentic evaluation, prompt-to-temporal specification, and verification-guided repair all predate this work.

`PhyReAct` is a grounded instance of this ReAct family specialized for physical verification. Rather than let perception decide the verdict in prompt space, the reasoning model compiles a prompt into *typed physical obligations*, and frozen physical operators—counting, tracking, segmentation, depth, OCR, audio, and open-vocabulary semantics—return measurements rather than judgments. Observations then gate and scope the obligations that remain worth measuring, and a fixed set of composition rules rolls the measurements into a

three-valued state (**supported**, **contradicted**, or **unknown**, surfaced as **abstain** when evidence is insufficient) while retaining claim-level provenance. This plan-before-act discipline and its code-as-plan compilation inherit directly from ReWOO, LLMCompiler, and ViperGPT [28, 57, 64]: the control flow follows the plan–execute branch of ReAct, where observations gate and scope already-declared measurements rather than trigger free-form re-planning or the on-the-fly invention of new tool sequences.

Where `PhyReAct` currently departs from the ReAct lineage is memory: classic ReAct agents accumulate experience within a single episode, whereas `PhyReAct` today treats each clip independently. We view cross-clip, self-evolving memory as the extension the framework is built toward (section 6.6) and survey the relevant literature separately in section 8.4.

These positions clarify our relationship to the closest neuro-symbolic evaluators. NeuS-V offers no formal guarantee when its learned perception is wrong [52], which is precisely why `PhyReAct` keeps measurements separate from verdicts and preserves per-claim provenance, so that a wrong measurement remains locally attributable rather than silently absorbed into a score. NeuS-E [11], and more broadly Reflexion [54] and CRITIC [17], produce diagnoses but stop short of turning them into autonomous, online correction of the underlying generator; converting our audited critiques into such a control channel (section 4.8) is a future direction we outline rather than an implemented result.

8.4 Agent memory, context-training, and self-evolving agents

Because `PhyReAct` is designed to evolve by optimizing a human-readable state $S = (\Pi, M, K)$ rather than by updating weights (section 6), it inherits from a fast-growing literature on where an agent's experience should live and how it should be revised. We organize that literature along the three components of S and, orthogonally, along the storage medium.

Where experience is stored: explicit state versus weights. A central question is whether accumulated experience should be internalized into parameters or externalized into a readable store. Evidence that explicit, retrievable memory can outperform implicit weight-internalization on multi-hop reasoning [77] motivates our commitment to a weight-free state: the reasoning model and physical operators stay frozen, and all learning is an edit to Π, M, or K. Context-training makes this precise by treating deployment-time adaptation as optimization of a textual, human-readable context rather than gradient descent [25], and self-evolving reasoning memories such as ReasoningBank show that curating what an agent remembers—distilled from both successful and failed trajectories—improves it without any parameter update [47, 54]. Surveys of agent memory chart this design space and the accompanying management lifecycle [32, 76].

Procedural memory (Π): reusable operators and programs. The operator library Π is a procedural memory in the sense of PlugMem, which distills raw trajectories into reusable, provenance-carrying procedures indexed for retrieval [67], and of the program-of-thought agents whose chains of thought are executable compositions of tool calls [19, 28, 57, 64]. `PhyReAct` specializes this to physical measurement: each compiled check is a cost-annotated closure that can, in principle, be named and reused across clips.

Episodic and semantic memory (M): distilling and refining priors. The experience library M corresponds to episodic and semantic memory. A-Mem maintains a persistent, self-organizing episodic store of past interactions [65]; ViLoMem distills multimodal experience through grow-and-refine error attribution with no gradient updates [7]; and M3-Agent, the closest analogue in the video domain, builds an entity-centric multimodal memory graph over per-clip observations [39]. We adopt Hindsight's discipline of separating recorded *evidence* from derived *belief*, so that traceable updates never silently overwrite the raw observations an auditor may later revisit [29].

External knowledge (K) and pollution control. Because a loop that recycles only its own conclusions can amplify its biases, K admits external knowledge, but under a candidate-and-pruning discipline with a do-nothing fallback. This follows the finding that naively equipping an agent with retrieval can degrade it through context pollution, and that maintaining and pruning candidate contexts against a held-out signal is what turns external information into consistent gains [25].

Positioning. Two properties distinguish `PhyReAct` within this literature. First, the *carrier* of memory is a structured physical-verification trace with full provenance and a three-valued verdict, rather than free-text notes or latent vectors, which is what makes the accumulated state auditable by construction. Second, we are explicit that none of this cross-clip machinery is run here: every memory citation above is a directional precedent for the roadmap of section 7, not evidence that `PhyReAct` already carries state across clips, and any distilled prior would be a critic-derived summary rather than ground truth.

8.5 Diagnostic supervision for generated-video failures

Human supervision for generated video ranges from holistic ratings and preferences to dimension-level explanations. VideoGen-Eval evaluates human agreement over more than 12,000 videos generated from 700 structured prompts [70]. Physion-Eval is closer to flaw diagnosis: its 12,718 generated videos and 10,990 expert reasoning traces cover 22 physical categories, temporally localize failures, and use corresponding real-world videos as references [74].

Our 1,500-clip corpus is complementary rather than a replacement. Its 2,582 human-written records bind each flaw to the violated prompt fragment and a rationale, severity, and confidence, with an optional time span and box track. This schema supports open-vocabulary allegation-to-flaw matching for a prompt-conditioned critic without a matched real-world reference. The resulting benchmark is more directly aligned with `PhyReAct`'s output contract, but its all-flawed, single-annotator construction supports recall measurement only and does not provide the precision or agreement statistics of a balanced, redundantly annotated benchmark.

9 Limitations

Generation. The control signal comes from a simulator, so scene diversity is bounded by what is simulated, and the mapping from simulation to prompt appearance is not itself verified for physical consistency.

Benchmark. The corpus is recall-only (no clean clips), so precision is not directly measurable; flaws are single-annotator with no inter-annotator agreement; the scope/category tags are machine-derived; and the source generator of the clips is unrecorded. Repeatedly designing against one frozen set risks overfitting to it even without training on it.

Critic. The spatial relation predicate and negative-event verification are limited; the typed-plan layer does not yet dispatch the full specialist registry; binding remains a central failure mode, since the compiler enforces symbolic identifier consistency but cannot guarantee that heterogeneous tools localized the same physical instance; and the reasoner-mediated sub-checks reduce but do not remove semantic opacity.

No cross-clip memory or self-evolution. `PhyReAct` is stateless across clips: each clip is planned and executed independently, so the system cannot reuse a verified claim-to-plan mapping for a similar prompt, nor accumulate the recurring physics-failure modes it observes (for example, that a requested multi-object collision is routinely mis-generated). Section 6.6 sketches the intended remedy—a context-trained experience library that distills each fully-provenanced trace into human-readable, versionable priors without updating weights [25, 47, 77], grows the operator set through multimodal distillation [7, 39, 67], and breaks its internal loop by admitting external knowledge only under a candidate-and-prune discipline [25]. That design is entirely future work: retrieval quality, staleness of cached plans, and the propagation of a wrong prior remain open, and any distilled prior would be *critic-derived*, not ground truth. Because every trace is already serialized with full provenance (section 4.6), the evidence records are a ready substrate for such a store, but we neither build nor evaluate one here.

10 Conclusion

Physical faithfulness in generated video requires three things that are usually studied apart: a way to *steer* what gets generated, a way to *measure* whether physical obligations are met, and *ground truth* against which measurements can be scored. This report contributes one artifact toward each. For steering, we build a

simulation-driven controllable generator (MuJoCo → Wan 2.2-VACE), a first step toward the control channel through which a critic might one day feed corrections back into generation. For ground truth, we assemble a benchmark of human-annotated, localized defects, so that a critic can be scored on whether it finds the specific fault rather than on aggregate preference. And for measurement, we present `PhyReAct`, an auditable multi-tool critic that grounds the reason–act–observe loop in physical verification: reasoning compiles the prompt into typed physical checks, frozen expert operators are called only to return measurements over the scope those checks declare, and a deterministic roll-up folds the observations into a three-valued verdict (**supported**/**contradicted**/**unknown**, surfaced as *plausible*, *implausible*, or **abstain**) with a human-traceable evidence chain.

Our central empirical result is narrow and stated as such: against human labels, on a development core, plan-grounded execution recalls more of the localized defects than per-claim verification does, at a modestly higher operator-call cost. This core is recall-only, singly annotated, and not held out—the system was developed on it—so an equal-cost control and a held-out evaluation both remain to be run before the comparison can be trusted beyond this setting.

The trajectory the framing implies is clear even where the experiments are not yet done. The traceable evidence `PhyReAct` already emits is the raw material for weight-free self-evolution through context-training, for growing the operator library as new measurement procedures are distilled, and—once the control channel is closed—for turning critic verdicts into generator corrections. Closing that critic → generator loop is the next milestone; we do not claim to have closed it here, nor that the generator has been made more physically faithful as a result. What we claim is a grounded, auditable place to start.

A Appendix

To be expanded: the full generation configuration and control-render details; the benchmark schema and per-stratum statistics; the critic's plan schema, predicate type rules, and temporal-relation truth tables; the specialist license and revision manifest; and the pre-registered evaluation manifest.

References

[1] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434.

[2] Niki Amini-Naieni and Andrew Zisserman. Open-world object counting in videos. In *AAAI*, 2026. arXiv:2506.15368. The CountVid model; the count arm.

[3] Niki Amini-Naieni, Tengda Han, and Andrew Zisserman. Countgd: Multi-modal open-world counting. In *NeurIPS*, 2024. arXiv:2407.04619.

[4] Xiangyu Bai, He Liang, Bishoy Galoaa, Utsav Nandi, Shayda Moezzi, Yuhang He, and Sarah Ostadabbas. Moregen: Multi-agent motion-reasoning engine for code-based text-to-video synthesis. In *CVPR*, pages 7632–7642, 2026. arXiv:2512.04221.

[5] Hritik Bansal, Zongyu Lin, Tianyi Xie, Zeshun Zong, Michal Yarom, Yonatan Bitton, Chenfanfu Jiang, Yizhou Sun, Kai-Wei Chang, and Aditya Grover. Videophy: Evaluating physical commonsense for video generation. *arXiv preprint arXiv:2406.03520*, 2024.

[6] Hritik Bansal, Clark Peng, Yonatan Bitton, Roi Goldenberg, Aditya Grover, and Kai-Wei Chang. Videophy-2: A challenging action-centric physical commonsense evaluation in video generation. *arXiv preprint arXiv:2503.06800*, 2025.

[7] Weihao Bo, Shan Zhang, Yanpeng Sun, Jingjing Wu, Qunyi Xie, Xiao Tan, Kunbin Chen, Wei He, Xiaofan Li, Na Zhao, Jingdong Wang, and Zechao Li. ViLoMem: Agentic learner with grow-and-refine multimodal semantic memory. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2511.21678. Dual-stream visual/logical error distillation, grow-and-refine (merge/create), no weight update. (metadata per project notes; verify before submission).

[8] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Dídac Surís, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. Open-vocabulary concept segmentation/tracking; used by the relation depth arm.

[9] Yongfan Chen, Xiuwen Zhu, and Tianyu Li. A physical coherence benchmark for evaluating video generation models via optical flow-guided frame prediction. *arXiv preprint arXiv:2502.05503*, 2025.

[10] Jaemin Cho, Yushi Hu, Roopal Garg, Peter Anderson, Ranjay Krishna, Jason Baldridge, Mohit Bansal, Jordi Pont-Tuset, and Su Wang. Davidsonian scene graph: Improving reliability in fine-grained evaluation for text-to-image generation. In *ICLR*, 2024. arXiv:2310.18235.

[11] Minkyu Choi, S P Sharan, Harsh Goel, Sahil Shah, and Sandeep Chinchali. We'll fix it in post: Improving text-to-video generation with neuro-symbolic feedback. *arXiv preprint arXiv:2504.17180*, 2025.

[12] Rohan Choudhury, Koichiro Niinuma, Kris M. Kitani, and László A. Jeni. Video question answering with procedural programs. In *ECCV*, pages 315–332, 2024. arXiv:2312.00937; the video analogue of ViperGPT.

[13] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. In *ECCV*, 2024. arXiv:2403.11481.

[14] Lin Geng Foo, Mark He Huang, Alexandros Lattas, Stylianos Moschoglou, Thabo Beeler, and Christian Theobalt. Physical simulator in-the-loop video generation. In *CVPR*, pages 4301–4311, 2026. arXiv:2603.06408.

[15] Shenghao Fu, Qize Yang, Qijie Mo, Junkai Yan, Xihan Wei, Jingke Meng, Xiaohua Xie, and Wei-Shi Zheng. Llmdet: Learning strong open-vocabulary object detectors under the supervision of large language models. In *CVPR*, 2025. arXiv:2501.18954.

[16] Hiroki Furuta, Heiga Zen, Dale Schuurmans, Aleksandra Faust, Yutaka Matsuo, Percy Liang, and Sherry Yang. Improving dynamic object interactions in text-to-video generation with ai feedback. *arXiv preprint arXiv:2412.02617*, 2024.

[17] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive critiquing. In *ICLR*, 2024. arXiv:2305.11738.

[18] Jing Gu, Xian Liu, Yu Zeng, Ashwin Nagarajan, Fangrui Zhu, Daniel Hong, Yue Fan, Qianqi Yan, Kaiwen Zhou, Ming-Yu Liu, and Xin Eric Wang. Phyworldbench: A comprehensive evaluation of physical realism in text-to-video models. In *ICLR*, 2026. Oral; arXiv:2507.13428.

[19] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *CVPR*, 2023. arXiv:2211.11559.

[20] Jiarui Hai, Helin Wang, Weizhe Guo, and Mounya Elhilali. Flexsed: Towards open-vocabulary sound event detection. *arXiv preprint arXiv:2509.18606*, 2025. Text-queryable sound-event detection with timestamps; the audio arm.

[21] Xuan He, Dongfu Jiang, Ge Zhang, Max Ku, Achint Soni, et al. Videoscore: Building automatic metrics to simulate fine-grained human feedback for video generation. In *EMNLP*, 2024. arXiv:2406.15252.

[22] Xuan He, Dongfu Jiang, Ping Nie, Minghao Liu, Zhengxuan Jiang, et al. Videoscore2: Think before you score in generative video evaluation. *arXiv preprint arXiv:2509.22799*, 2025.

[23] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. CLIPScore: A reference-free evaluation metric for image captioning. In *EMNLP*, 2021. arXiv:2104.08718.

[24] Yushi Hu, Benlin Liu, Jungo Kasai, Yizhong Wang, Mari Ostendorf, Ranjay Krishna, and Noah A. Smith. TIFA: Accurate and interpretable text-to-image faithfulness evaluation with question answering. In *ICCV*, 2023. arXiv:2303.11897.

[25] Zeyu Huang, Adhiguna Kuncoro, Qixuan Feng, Jiajun Shen, Lucio Dery, Arthur Szlam, and Marc'Aurelio Ranzato. Context training with active information seeking. *arXiv preprint arXiv:2605.13050*, 2026. Learning as state optimization: edits a human-readable context, weights frozen; candidate-and-prune discipline against context pollution. (metadata per project notes; verify before submission).

[26] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *CVPR*, 2024. arXiv:2311.17982.

[27] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. Vace: All-in-one video creation and editing. *arXiv preprint arXiv:2503.07598*, 2025. Mask-conditioned controllable video generation; the project's stylizer.

[28] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *ICML*, 2024. arXiv:2312.04511. Plans a DAG of function calls, dispatches independent calls in parallel.

[29] Chris Latimer, Nicoló Boschi, Chris Bartholomew, Andrew Neeser, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2026. Separates observed evidence from inferred belief; confidence-reinforced, traceable updates. (metadata per project notes; verify before submission).

[30] Daeun Lee, Jaehong Yoon, Jaemin Cho, and Mohit Bansal. Self-correcting text-to-video generation with misalignment detection and localized refinement. In *Findings of ACL*, pages 36464–36489, 2026. doi: 10.18653/v1/2026.findings-acl.1817.

[31] Keliang Li, Yansong Li, Hongze Shen, Mengdi Liu, Hong Chang, and Shiguang Shan. Lenswalk: Agentic video understanding by planning how you see in videos. In *CVPR*, pages 19518–19528, 2026. arXiv:2603.24558.

[32] Jiafeng Liang, Hao Li, Chang Li, Jiaqi Zhou, Shixin Jiang, Zekun Wang, Changkai Ji, Zhihao Zhu, Runxuan Liu, Tao Ren, Jinlan Fu, See-Kiong Ng, Xia Liang, Ming Liu, and Bing Qin. AI meets brain: Memory systems from cognitive neuroscience to autonomous agents. *arXiv preprint arXiv:2512.23343*, 2026. Survey aligning cognitive-neuroscience memory with LLM/agent memory; expert-model symbolization of dense frames, procedural/strategic self-evolution. (metadata per project notes; verify before submission).

[33] Zhiqiu Lin, Deepak Pathak, Baiqi Li, Jiayao Li, Xide Xia, Graham Neubig, Pengchuan Zhang, and Deva Ramanan. Evaluating text-to-visual generation with image-to-text generation. In *ECCV*, 2024. VQAScore. arXiv:2404.01291.

[34] Fangfu Liu, Hanyang Wang, Yimo Cai, Kaiyan Zhang, Xiaohang Zhan, and Yueqi Duan. Video-t1: Test-time scaling for video generation. In *ICCV*, pages 18671–18681, 2025.

[35] Shaowei Liu, Zhongzheng Ren, Saurabh Gupta, and Shenlong Wang. Physgen: Rigid-body physics-grounded image-to-video generation. In *ECCV*, pages 360–378, 2024.

[36] Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tieyong Zeng, Raymond Chan, and Ying Shan. EvalCrafter: Benchmarking and evaluating large video generation models. In *CVPR*, 2024. arXiv:2310.11440.

[37] Yuanxin Liu, Lei Li, Shuhuai Ren, Rundong Gao, Shicheng Li, Sishuo Chen, Xu Sun, and Lu Hou. FETV: A benchmark for fine-grained evaluation of open-domain text-to-video generation. In *NeurIPS*, 2023. arXiv:2311.01813.

[38] Zhaoyang Liu, Zeqiang Lai, Zhangwei Gao, Erfei Cui, Ziheng Li, et al. Controllllm: Augment language models with tools by searching on graphs. *arXiv preprint arXiv:2310.17796*, 2023. Tool use as search over a tool graph ("Thoughts-on-Graph").

[39] Lin Long, Yichen He, Wentao Ye, Yiyuan Pan, Yuan Lin, Hang Li, Junbo Zhao, and Wei Li. Seeing, listening, remembering, and reasoning: A multimodal agent with long-term memory. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2508.09736. M3-Agent: entity-centric multimodal memory graph over per-clip observations with multi-turn retrieval. (metadata per project notes; verify before submission).

[40] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2022.

[41] Fanqing Meng, Jiaqi Liao, Xinyu Tan, Wenqi Shao, Quanfeng Lu, Kaipeng Zhang, Yu Cheng, Dianqi Li, Yu Qiao, and Ping Luo. Towards world simulator: Crafting physical commonsense-based benchmark for video generation. *arXiv preprint arXiv:2410.05363*, 2024. The PhyGenBench physical-commonsense generation benchmark.

[42] Sachit Menon, Ahmet Iscen, Arsha Nagrani, Tobias Weyand, Carl Vondrick, and Cordelia Schmid. Caviar: Critic-augmented video agentic reasoning. *arXiv preprint arXiv:2509.07680*, 2025.

[43] Juhong Min, Shyamal Buch, Arsha Nagrani, Minsu Cho, and Cordelia Schmid. Morevqa: Exploring modular reasoning models for video question answering. In *CVPR*, 2024. arXiv:2404.06511.

[44] Antonio Montanaro, Luca Savant Aira, Emanuele Aiello, Diego Valsesia, and Enrico Magli. Motioncraft: Physics-based zero-shot video generation. In *NeurIPS*, 2024.

[45] Saman Motamed, Laura Culp, Kevin Swersky, Priyank Jaini, and Robert Geirhos. Do generative video models understand physical principles? *arXiv preprint arXiv:2501.09038*, 2025. The Physics-IQ benchmark; visual realism does not imply physical understanding.

[46] Sriram Narayanan, Ziyu Jiang, Srinivasa Narasimhan, and Manmohan Chandraker. Phyco: Learning controllable physical priors for generative motion. In *CVPR*, pages 41892–41902, 2026. arXiv:2604.28169.

[47] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.25140. Distills reusable reasoning strategies across episodes.

[48] Qwen Team. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. The vision-language reasoner (Qwen3-VL-30B-A3B-Instruct) at the centre of the critic.

[49] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhobe. Efficient and extensible algorithms for multi query optimization. In *SIGMOD*, 2000. Practical shared-subexpression MQO in a Volcano-style optimizer.

[50] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023. arXiv:2302.04761.

[51] Timos K. Sellis. Multiple-query optimization. *ACM Transactions on Database Systems (TODS)*, 13(1):23–52, 1988. Foundational MQO: optimize a batch of queries jointly via shared sub-expressions.

[52] S P Sharan, Minkyu Choi, Sahil Shah, Harsh Goel, Mohammad Omama, and Sandeep Chinchali. Neuro-symbolic evaluation of text-to-video models using formal verification. In *CVPR*, pages 8395–8405, 2025. arXiv:2411.16718.

[53] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. In *NeurIPS*, 2023. arXiv:2303.17580.

[54] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023. arXiv:2303.11366.

[55] Sanjay Subramanian, Medhini Narasimhan, Kushal Khangaonkar, Kevin Yang, Arsha Nagrani, Cordelia Schmid, Andy Zeng, Trevor Darrell, and Dan Klein. Modular visual question answering via code generation. In *ACL*, 2023. arXiv:2306.05392.

[56] Kaiyue Sun, Kaiyi Huang, Xian Liu, Yue Wu, Zihan Xu, Zhenguo Li, and Xihui Liu. T2V-CompBench: A comprehensive benchmark for compositional text-to-video generation. *arXiv preprint arXiv:2407.14505*, 2024.

[57] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *ICCV*, 2023. arXiv:2303.08128.

[58] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[59] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. The Wan 2.1 open video-diffusion backbone.

[60] Qian Wang, Ziqi Huang, Ruoxi Jia, Paul Debevec, and Ning Yu. MAViS: A multi-agent framework for long-sequence video storytelling. *arXiv preprint arXiv:2508.08487*, 2025. Explore-Examine-Enhance closed loop: evaluator feedback rewrites prompts and regenerates. Generator-side self-evaluation. (metadata per project notes; verify before submission).

[61] Xiang Wang, Hangjie Yuan, Shiwei Zhang, Dayou Chen, Jiuniu Wang, Yingya Zhang, Yujun Shen, Deli Zhao, and Jingren Zhou. VideoComposer: Compositional video synthesis with motion controllability. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[62] Xiaohan Wang, Yuhui Zhang, Orr Zohar, and Serena Yeung-Levy. Videoagent: Long-form video understanding with large language model as agent. In *ECCV*, 2024. arXiv:2403.10517.

[63] Ziyang Wang, Shoubin Yu, Elias Stengel-Eskin, Jaehong Yoon, Feng Cheng, Gedas Bertasius, and Mohit Bansal. Videotree: Adaptive tree-based video representation for llm reasoning on long videos. In *CVPR*, 2025. arXiv:2405.19209.

[64] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023. Plan-up-front, then execute; the ReWOO plan-execute separation.

[65] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.12110. Persistent, self-organizing episodic store.

[66] Qiyao Xue, Xiangyu Yin, Boyuan Yang, and Wei Gao. Phyt2v: Llm-guided iterative self-refinement for physics-grounded text-to-video generation. In *CVPR*, pages 18826–18836, 2025. arXiv:2412.00596.

[67] Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. PlugMem: A task-agnostic plugin memory module for LLM agents. *arXiv preprint arXiv:2603.03296*, 2026. Distills trajectories into a knowledge-centric graph of propositional facts and prescriptive workflows with provenance edges. (metadata per project notes; verify before submission).

[68] Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. Gpt4tools: Teaching large language model to use tools via self-instruction. In *NeurIPS*, 2023. arXiv:2305.18752.

[69] Xindi Yang, Baolu Li, Yiming Zhang, Zhenfei Yin, Lei Bai, Liqian Ma, Zhiyong Wang, Jianfei Cai, Tien-Tsin Wong, Huchuan Lu, and Xu Jia. Vlipp: Towards physically plausible video generation with vision and language informed physical prior. In *ICCV*, pages 12360–12370, 2025. arXiv:2503.23368.

[70] Yuhang Yang, Ke Fan, Shangkun Sun, Hongxiang Li, Ailing Zeng, FeiLin Han, Wei Zhai, Wei Liu, Yang Cao, and Zheng-Jun Zha. Videogen-eval: Agent-based system for video generation evaluation. *arXiv preprint arXiv:2503.23452*, 2025.

[71] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023. arXiv:2210.03629.

[72] Ke Zhang, Cihan Xiao, Jiacong Xu, Yiqun Mei, and Vishal M. Patel. Think before you diffuse: Infusing physical rules into video diffusion. *arXiv preprint arXiv:2505.21653*, 2025. The parent "DiffPhy" work this project extends. Title as verified live on the arXiv abstract page; some project docs cite an earlier subtitle "Physics-Aware Video Generation via Chain-of-Thought Conditioning.".

[73] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[74] Qin Zhang, Peiyu Jing, Hong-Xing Yu, Fangqiang Ding, Fan Nie, Weimin Wang, Yilun Du, James Zou, Jiajun Wu, and Bing Shuai. Physion-eval: Evaluating physical realism in generated video via human reasoning. *arXiv preprint arXiv:2603.19607*, 2026.

[75] Yongdong Zhang, Yi Wang, Yixiao Ge, Yuying Ge, Xintao Li, Ying Shan, and Xiaohu Wang. Timelens: Rethinking video temporal grounding with multimodal llms. *arXiv preprint arXiv:2512.14698*, 2025. Closest published reference for the deployed action arm (the MCG-NJU TimeLens2-8B checkpoint has no standalone paper).

[76] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024. Landscape reference for agent memory; PhyReAct maintains none.

[77] Zeyu Zhang, Yang Zhang, Haoran Tan, Rui Li, and Xu Chen. Explicit v.s. implicit memory: Exploring multi-hop complex reasoning over personalized information. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2026. arXiv:2508.13250. Evidence that explicit external retrievable memory outperforms implicit weight-internalized memory. (metadata per project notes; verify before submission).

[78] Artem Zholus, Carl Doersch, Yi Yang, Skanda Koppula, Viorica Patraucean, Xu He, Ignacio Rocco, Mehdi S. M. Sajjadi, Sarath Chandar, and Ross Goroshin. Tapnext: Tracking any point (tap) as next token prediction. *arXiv preprint arXiv:2504.05579*, 2025. The point-tracking backbone behind the kinematic (physics) arm's TAPNext++.

[79] Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A. Rossi, Somdeb Sarkhel, and Chao Zhang. Toolchain*: Efficient action space navigation in large language models with a* search. In *ICLR*, 2024. arXiv:2310.13227.